



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
LALITPUR ENGINEERING COLLEGE**

NETWORK INTRUSION DETECTION SYSTEM

BY

AASTHA BHATT [LEC-080-BCT-002]

DHARMENDRA MANDAL [LEC-080-BCT-013]

KIRAN THAPA MAGAR [LEC-080-BCT-017]

RINESH SHAKYA [LEC-080-BCT-026]

**A THIRD YEAR PROJECT REPORT
SUBMITTED TO THE DEPARTMENT OF COMPUTER ENGINEERING
IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR
THE DEGREE OF BACHELOR OF ENGINEERING IN COMPUTER
ENGINEERING**

**DEPARTMENT OF COMPUTER ENGINEERING
LALITPUR, NEPAL**

AUGUST, 2026



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
LALITPUR ENGINEERING COLLEGE**

**A Project Report
on
Network Intrusion Detection System**

**Submitted By
AASTHA BHATT [LEC-080-BCT-002]
DHARMENDRA MANDAL [LEC-080-BCT-013]
KIRAN THAPA MAGAR [LEC-080-BCT-017]
RINESH SHAKYA [LEC-080-BCT-026]**

**Submitted To:
Department of Computer Engineering
Lalitpur Engineering College
Lalitpur, Nepal**

**In partial fulfillment for the award of Bachelor of Engineering in Computer
Engineering**

AUGUST, 2026

ACKNOWLEDGMENT

This project work would not have been possible without the guidance and the help of several individuals who in one way or another contributed and extended their valuable assistance in the preparation and completion of this study.

First of all, We would like to express our sincere gratitude to our supervisor, **Er. Johnson Timalsina** for providing invaluable guidance, insightful comments, meticulous suggestions, and encouragement throughout the duration of this project work. Our sincere thanks also goes to the Project Coordinator, **Er. Sandesh Sharan Poudel**, for coordinating the project works, providing astute criticism, and having inexhaustible patience.

We would also like to offer our gratitude to **Er. Bibat Thokar** whose valuable suggestions helped this project reach the expected goal.

We are also grateful to our classmates and friends for offering us advice and moral support. To our family, thank you for encouraging us in all of our pursuits and inspiring us to follow our dreams. We are especially grateful to our parents, who supported us emotionally, believed in us and wanted the best for us.

AASTHA BHATT [LEC-080-BCT-002]

DHARMENDRA MANDAL [LEC-080-BCT-013]

KIRAN THAPA MAGAR [LEC-080-BCT-017]

RINESH SHAKYA [LEC-080-BCT-026]

AUGUST, 2026

ABSTRACT

The rapid growth of digital networks and Internet-based services has increased the frequency and complexity of cyberattacks, making effective Network Intrusion Detection Systems (NIDS) essential for modern network security. Traditional intrusion detection approaches may struggle to identify evolving attacks and handle highly imbalanced network traffic. This project presents a Machine Learning-based Network Intrusion Detection System designed to classify benign and malicious network traffic using the CICIDS2017 dataset. The system applies data preprocessing, feature scaling, label encoding, and class balancing using the SMOTETomek technique to reduce the effect of class imbalance. Four machine learning models, namely Naive Bayes, Random Forest, XGBoost, and Long Short-Term Memory (LSTM), were implemented and evaluated using accuracy, precision, recall, F1-score, and false positive rate. The experimental results show that XGBoost achieved the best overall performance with an accuracy of approximately 99.87%, followed by Random Forest with approximately 99.83%. LSTM also achieved strong performance with approximately 98.62%, while Naive Bayes provided comparatively lower performance. The results demonstrate that ensemble-based machine learning models, particularly XGBoost and Random Forest, are highly effective for CICIDS2017-based intrusion detection. The proposed system also provides mitigation recommendations for detected attack categories, supporting users in responding to potential network threats. Overall, the findings demonstrate the effectiveness of machine learning techniques for accurate and reliable network intrusion detection.

Keywords: *CICIDS2017, Network Intrusion Detection System, XGBoost, Random Forest, Naive Bayes, LSTM, SMOTETomek, Machine Learning*

TABLE OF CONTENTS

ACKNOWLEDGMENT	iii
ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF FIGURES	viii
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	1
1.4 Project Objectives	2
1.5 Scope and application of Project	2
1.5.1 Scope of Project	2
1.5.2 Application of Project	2
1.6 Originality of Project	3
1.7 Organization of Project Report	3
2 LITERATURE REVIEW	5
2.1 Related works	5
2.2 Limitation of Previous works	6
2.3 Significance of the Project	7
2.4 Comparison of Previous works	8
3 REQUIREMENT ANALYSIS	9
3.1 Hardware Requirements	9
3.2 Software Requirements	9
3.3 Functional Requirements	10
3.4 Non-Functional Requirements	10
3.5 Feasibility Study	10
3.5.1 Technical Feasibility	10
3.5.2 Economic Feasibility	11

3.5.3	Operational Feasibility	11
3.5.4	Legal Feasibility	11
4	System Design	12
4.1	System Block Diagram	12
4.2	Block Descriptions	13
4.2.1	Dataset	13
4.2.2	Data Preprocessing	13
4.2.3	Feature Selection	13
4.2.4	Model Training	14
4.2.5	Trained Model	14
4.2.6	Intrusion Detection	14
4.2.7	Output and Mitigation	15
4.3	Data Flow Diagram	15
4.3.1	DFD Level 0	15
4.3.2	DFD Level 1	16
5	METHODOLOGY	17
5.1	Overview	17
5.2	Detailed Description of Each Stage	17
5.2.1	Dataset Selection	17
5.2.2	Data Preprocessing	17
5.2.3	Feature Selection	20
5.2.4	Model Training	20
5.2.5	Model Selection	21
5.2.6	Model Testing and Prediction	25
5.2.7	Performance Evaluation	26
5.2.8	Mitigation Methods	27
5.3	Results	29
5.3.1	Experimental Setup	29
5.3.2	Results Before Applying SMOTETomek	29
5.3.3	Comparison of Machine Learning Models	33
5.3.4	Results After Applying SMOTETomek	37
5.3.5	Comparison After Applying SMOTETomek	39

5.3.6	Effect of SMOTETomek on Model Performance	42
5.3.7	Selection of XGBoost	43
5.3.8	Minority Attack Detection	43
5.3.9	Confusion Matrix Analysis	45
5.3.10	Final Model Performance	47
5.3.11	Final System User Interface	48
6	Discussion and analysis	49
6.0.1	Discussion and Analysis	49
7	Conclusion and future enchancement	51
7.1	Conclusion	51
7.2	Limitations of the Project	51
7.3	Future Enhancements	52
7.4	Ethics of AI	52
APPENDIX A		
A.1	Report Schedule (Gantt Chart)	54
REFERENCES		56

LIST OF FIGURES

Figure 1.1	Organisation of Project Report	4
Figure 4.1	System Architecture	12
Figure 4.2	DFD Level 0-Context Diagram of NIDS	15
Figure 4.3	DFD Level 1-Machine Learning Based NIDS flow	16
Figure 5.1	Methodology of the Proposed Network Intrusion Detection System ..	17
Figure 5.2	Naive Bayes classification report before SMOTETomek	30
Figure 5.3	Random Forest classification report before SMOTETomek	30
Figure 5.4	XGBoost classification report before SMOTETomek	31
Figure 5.5	LSTM classification report	32
Figure 5.6	LSTM Accuracy Graph	32
Figure 5.7	LSTM Loss Graph	33
Figure 5.8	Accuracy comparison of machine learning models before SMOTE- Tomek	34
Figure 5.9	precision comparison of machine learning models before SMOTE- Tomek	34
Figure 5.10	Recall comparison of machine learning models before SMOTETomek	35
Figure 5.11	F1 comparison of machine learning models before SMOTETomek ...	35
Figure 5.12	Training time comparison of machine learning models before SMOTE- Tomek	36
Figure 5.13	Prediction time comparison of machine learning models before SMOTE- Tomek	36
Figure 5.14	Naive Bayes classification report after SMOTETomek	37
Figure 5.15	Random Forest classification report after SMOTETomek	38
Figure 5.16	XGBoost classification report after SMOTETomek	39
Figure 5.17	Accuracy comparison of machine learning models after SMOTETomek	40
Figure 5.18	precision comparison of machine learning models after SMOTETomek	40
Figure 5.19	Recall comparison of machine learning models after SMOTETomek .	41
Figure 5.20	F1 comparison of machine learning models after SMOTETomek	41
Figure 5.21	Training time comparison of machine learning models after SMOTE- Tomek	42

Figure 5.22 Prediction time comparison of machine learning models after SMOTE-Tomek	42
Figure 5.23 Comparison of minority attack detection before and after SMOTE-Tomek	43
Figure 5.24 Per-Class detection before SmoteTomek.....	44
Figure 5.25 Per-Class detection after SmoteTomek	44
Figure 5.26 Confusion matrices of the Naives Bayes after SMOTETomek	45
Figure 5.27 Confusion matrices of the Random Forest after SMOTETomek	46
Figure 5.28 Confusion matrices of the XGBoost after SMOTETomek	46
Figure 5.29 Confusion matrices of the LSTM after SMOTETomek	47
Figure 5.30 System User Interface	48
Figure 5.31 Single Attack Detection Interface	48
Figure 5.32 Batch Attack Detection Interface.....	48
Figure A.1 8-Week Gantt Chart for NIDS Project	54

LIST OF TABLES

Table 2.1	Comparison of Existing NIDS Research Works	8
Table 5.1	Dataset Splitting Ratio	21
Table 5.2	Notation used in the LSTM model.....	25
Table 5.3	Attack Types and Implemented Mitigation Methods	28
Table 5.4	Model Performance before SMOTETomek	33
Table 5.5	Performance Comparison of Models after SMOTETomek	39
Table 5.6	Comparison Before and After SMOTETomek	43

LIST OF ABBREVIATIONS

List of Abbreviations

AI	Artificial Intelligence
CICIDS2017	Canadian Institute for Cybersecurity Intrusion Detection System 2017
DDoS	Distributed Denial of Service
DoS	Denial of Service
PortScan	Port Scanning Attack
XSS	Cross-Site Scripting
SQL	Structured Query Language
FTP	File Transfer Protocol
SSH	Secure Shell
EDA	Exploratory Data Analysis
FN	False Negative
FP	False Positive
IDS	Intrusion Detection System
LSTM	Long Short-Term Memory
ML	Machine Learning
NB	Naive Bayes
NIDS	Network Intrusion Detection System
RF	Random Forest
SMOTE	Synthetic Minority Over-sampling Technique
SMOTETomek	SMOTE with Tomek Links
TN	True Negative
TP	True Positive
XGB	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Background

In today's highly interconnected digital world, computer networks play a vital role in enabling communication, data exchange, and the functioning of critical services across various domains such as banking, healthcare, education, and government systems. However, with the rapid expansion of internet-based services, cyber threats have also evolved in terms of frequency, scale, and sophistication. Cyberattacks such as denial-of-service (DoS), probing, user-to-root (U2R), and remote-to-local (R2L) attacks pose serious risks to the confidentiality, integrity, and availability of network resources. With the rapid evolution of cyber threats, traditional rule-based systems are no longer sufficient to provide robust security. This has led to the adoption of Machine Learning (ML) techniques in cybersecurity, particularly in the development of modern Network Intrusion Detection Systems (NIDS). Machine Learning enables systems to learn patterns from historical network data and automatically classify traffic as normal or malicious without relying solely on predefined rules. NIDSs are security solutions that, like antivirus software, firewalls, and access control schemes, are designed to make information and communication systems more secure.

1.2 Motivation

This project is motivated by a personal experience of a cyberattack in which sensitive personal information was compromised and multiple online accounts were affected. This incident highlighted the increasing risks and real-world impact of cyber threats in today's digital environment. As a result, it inspired the selection of this project topic with the aim of understanding cyberattack mechanisms and developing effective machine learning-based tools for detecting and preventing such attacks in the future.

1.3 Problem Statement

Traditional Intrusion Detection Systems (IDS) typically rely on signature-based and anomaly-based methods to detect and generate alerts for malicious network activity. However, these approaches often face difficulties in accurately identifying and classifying a wide range of evolving cyberattacks, particularly in scenarios where the dataset is imbalanced and certain attack types are significantly uncommon.

1.4 Project Objectives

The objectives of our project are:

- To develop a Machine Learning-based Network Intrusion Detection System for detecting and classifying malicious network activities.
- To classify attacks in the CICIDS2017 dataset and provide appropriate mitigation measures.

1.5 Scope and application of Project

1.5.1 Scope of Project

The scope of this project is to design and implement a Machine Learning-based Network Intrusion Detection System (NIDS) capable of analyzing network traffic data and identifying malicious activities using the CICIDS2017 dataset. The system focuses on detecting and classifying various attacks, including **DoS, DDoS, PortScan, Brute Force, Web Attacks, Botnet, Infiltration, and Heartbleed**, using supervised machine learning algorithms. The project is limited to dataset-based analysis and does not cover large-scale real-time deployment in production networks. It aims to improve intrusion detection performance and reduce false alarms through data preprocessing, class balancing, feature scaling, model training, and evaluation techniques.

1.5.2 Application of Project

- It can be used in organizational networks to detect and prevent unauthorized access and cyberattacks.
- It can help Internet Service Providers (ISPs) monitor network traffic and ensure secure communication.
- It can be applied in banking and financial systems to enhance security against cyber threats.
- It is useful in healthcare systems to protect sensitive patient data from intrusion and breaches.
- It can be used in e-commerce platforms to secure user transactions and prevent fraud.

1.6 Originality of Project

The originality of this project lies in the implementation of a Machine Learning-based approach for Network Intrusion Detection using the CICIDS2017 dataset and customized data preprocessing techniques. Unlike traditional signature-based systems, the proposed system uses data-driven learning methods to identify and classify different network attacks. The system focuses on attack categories such as DoS, DDoS, PortScan, Brute Force, Web Attacks, Botnet, Infiltration, and Heartbleed, and evaluates multiple machine learning models using standard performance metrics. Additionally, the project emphasizes effective handling of class imbalance through preprocessing and data balancing techniques such as SMOTETomek, along with model comparison to achieve reliable intrusion detection performance.

1.7 Organization of Project Report

Chapter 1 :Introduction: It deals with the introduction of the project along with the background, motivation, problem statement, project objectives, and scope of the project.

Chapter 2: Literature Review: It deals with all the literature articles and papers taken in account for the ideas and motivation to associate this project.

Chapter 3: Requirement Analysis: It deals with requirements analysis such as hardware and software requirements.

Chapter 4: System Design: It consists of all system architecture of the project.

Chapter 5: Methodology: It consists of working principle, algorithm,, expression and software development, testing and evaluation used for the development of project.

Chapter 6: Outcome: It deals with what the result of the project should be . All the output of various scenarios (best case and worst case analysis).

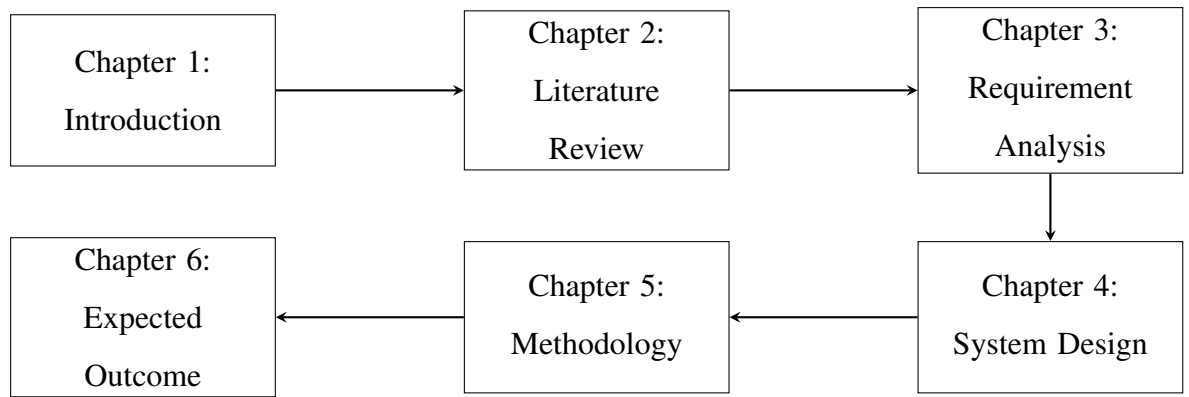


Figure 1.1: Organisation of Project Report

2 LITERATURE REVIEW

2.1 Related works

The literature review contains the previous or similar works done on the project, and these projects include:

Ravipati and Abualkibash (2019) evaluated several ML algorithms on KDD- 99 and NSL-KDD datasets. Random Forest achieved better accuracy, but the study lacked real-time implementation considerations [1].

Min et al. (2018) developed a hybrid IDS using Text-CNN for feature extraction and Random Forest for classification. The model improved detection performance but required high computational resources, making it less suitable for lightweight systems [2].

Maniriho and Ahmad (2018) compared multiple machine learning models including Random Forest and Naive Bayes for intrusion detection. Results showed that ensemble methods generally perform better, but attack-type classification was limited [3].

Obeidat et al. (2018) emphasized data preprocessing techniques such as normalization and feature selection to improve IDS performance. However, the study used the older KDD Cup 99 dataset, which reduces real-world applicability [4].

Al-Yaseen et al. (2023) compared several machine learning algorithms for network intrusion detection using the CICIDS2017 dataset. The study evaluated models including Random Forest, Support Vector Machine, Naive Bayes, and Logistic Regression, and applied SMOTE to address class imbalance. The results showed that ensemble-based methods achieved strong detection performance, demonstrating the importance of data balancing and model selection in intrusion detection [5].

Sapre et al. (2019) compared KDDCup99 and NSL-KDD datasets for IDS evaluation. They concluded that NSL-KDD is more balanced and suitable for machine learning-based intrusion detection[6].

Yin et al. (2017) proposed a deep learning approach for intrusion detection using Recurrent Neural Networks (RNN). The model was evaluated on network intrusion datasets

and demonstrated that deep learning could effectively learn patterns from network traffic for attack classification. However, the computational requirements of deep learning models can make them less suitable for resource-constrained environments [7].

Tripathy and Behera (2023) evaluated multiple ML models including Random Forest, XGBoost, and Naive Bayes. Ensemble methods showed superior performance, but the study used traditional datasets [8].

Shone et al. (2018) proposed a deep learning-based network intrusion detection system that combined non-symmetric deep autoencoders with a Random Forest classifier. The approach achieved improved detection performance compared with several traditional machine learning methods. However, the additional complexity of deep learning-based feature extraction can increase computational requirements [9].

Çavuşoğlu (2019) developed a hybrid intrusion detection approach using multiple machine learning methods and evaluated its performance for detecting network attacks. The study demonstrated that combining different learning techniques can improve intrusion detection performance. However, hybrid approaches may require additional computational resources and careful model configuration [10].

2.2 Limitation of Previous works

- Binary classification focus
- Poor handling of multi-class attacks
- Use of outdated datasets
- High computational complexity
- No model comparison or optimization

2.3 Significance of the Project

- The project improves upon previous NIDS approaches by combining machine learning-based attack detection with mitigation measures.
- It compares multiple machine learning models to identify an effective model for network intrusion detection.
- It addresses class imbalance in the CICIDS2017 dataset using data balancing techniques.
- It integrates attack prediction with a mitigation engine to provide appropriate responses to detected attacks.
- It also provides a user-friendly interface, making the system more practical for network security monitoring.

2.4 Comparison of Previous works

Table 2.1: Comparison of Existing NIDS Research Works

Ref	Methods Used	Dataset	Contribution	Limitations
[1]	Random Forest + Apache Spark	NSL-KDD	Distributed ML-based intrusion detection	Limited attack classification, high FN rate
[2]	Text-CNN + Random Forest	NSL-KDD	Hybrid deep learning + ML model	High computational complexity
[3]	Random Forest, Naive Bayes, SGD	NSL-KDD	Comparative ML performance analysis	Limited multi-class analysis
[4]	RF, NB, MLP, Bayes Network	KDD Cup 99	Impact of pre-processing techniques	Uses outdated dataset
[5]	RF, SVM, NB, LR	CICIDS2017	Compared ML models with SMOTE	Limited model and dataset scope
[6]	Multiple ML Algorithms	KDDCup99, NSL-KDD	Dataset comparison study	Focus only on datasets, not optimization
[7]	RNN	KDD-99, NSL-KDD	Applied deep learning to IDS	High computational cost
[8]	RF, XGBoost, KNN, SVM, NB	KDD-99	Evaluation of modern ML algorithms	Uses traditional datasets only
[9]	Deep Autoencoder + Random Forest	NSL-KDD	Combined deep feature extraction with Random Forest for NIDS	Increased model complexity and computational cost
[10]	Hybrid Machine Learning Methods	NSL-KDD	Improved intrusion detection through a hybrid ML approach	Requires careful model configuration and additional computational resources

3 REQUIREMENT ANALYSIS

3.1 Hardware Requirements

- Laptop or desktop computer
- Intel Core i3 processor or equivalent
- 4 GB RAM (minimum)
- 5 GB free storage space
- Network adapter (Wi-Fi or Ethernet)
- Keyboard and mouse/touchpad
- Internet connection

3.2 Software Requirements

- Windows 10/11 or Linux operating system
- Python 3.x
- Jupyter Notebook or Visual Studio Code (VS Code)
- Pandas library for data preprocessing and analysis
- NumPy library for numerical computations
- Scikit-learn library for machine learning model development
- Matplotlib library for data visualization
- Seaborn library for statistical data visualization
- CIC-CIDS017 dataset for training and testing the intrusion detection model
- Git (optional) for version control and project management

3.3 Functional Requirements

The proposed Machine Learning-based Network Intrusion Detection System shall be able to load and preprocess the CICIDS2017 dataset by handling missing values, encoding labels, and scaling numerical features. The system shall apply class balancing techniques such as SMOTETomek to address class imbalance and train multiple machine learning models, including Random Forest, XGBoost, LSTM, and Naive Bayes. It shall classify network traffic into normal and attack categories such as DoS, DDoS, PortScan, Brute Force, Web Attacks, Botnet, Infiltration, and Heartbleed. The system shall provide appropriate mitigation recommendations for detected attacks. It shall also perform prediction on unseen data and evaluate the models using accuracy, precision, recall, F1-score, and false positive rate. Additionally, the system shall compare the performance of the trained models to identify the most effective algorithm for intrusion detection.

3.4 Non-Functional Requirements

The system shall provide efficient processing and analysis of large network traffic datasets within a reasonable time. It shall maintain high detection performance while minimizing false positives and false negatives. The system shall be reliable and capable of producing consistent results during repeated executions. Furthermore, it shall be maintainable and extensible to support future improvements, including additional datasets and machine learning models. The system shall also be portable across standard computing environments using Python-based tools and libraries.

3.5 Feasibility Study

A feasibility study is conducted to determine whether the proposed system can be successfully developed within the available technical, economic, operational, and legal constraints.

3.5.1 Technical Feasibility

The proposed system is technically feasible because it uses widely available hardware and open-source software tools. Python libraries such as Scikit-learn, Pandas, NumPy, XGBoost, and TensorFlow provide the required functionality for data preprocessing, class balancing, model training, prediction, and evaluation. The CICIDS2017 dataset

is publicly available and provides network traffic data containing various benign and attack activities. Therefore, the required technologies and resources are accessible and sufficient for implementing the proposed NIDS.

3.5.2 Economic Feasibility

The project is economically feasible because it requires minimal financial investment. The primary software tools, including Python, Scikit-learn, Pandas, NumPy, XGBoost, TensorFlow, and Jupyter Notebook, are freely available or open-source. The system can be developed and tested using standard computer hardware without requiring specialized network monitoring equipment or expensive software licenses. Therefore, the overall development cost is low.

3.5.3 Operational Feasibility

The proposed system is operationally feasible because it can automatically analyze network traffic data and classify it as benign or malicious. The system can assist security administrators and researchers in identifying potential network attacks and understanding the type of detected threat. The inclusion of mitigation recommendations can further support users in taking appropriate security measures. However, the current implementation is primarily dataset-based and is not intended to replace a production-level real-time intrusion detection system.

3.5.4 Legal Feasibility

The project is legally feasible because it uses the publicly available CICIDS2017 dataset for research and educational purposes. The software libraries used in the project are distributed under their respective open-source licenses. The project does not involve unauthorized access to computer networks, collection of private user information, or execution of attacks against real systems. Therefore, no significant legal or compliance issues are expected during the development and evaluation of the system.

4 System Design

Our proposed system improves on previous NIDS approaches by combining data pre-processing, feature selection, machine learning-based detection, and mitigation in one system. While many previous works mainly focus on detecting attacks, our system also classifies different types of attacks and provides suitable mitigation measures. This makes the system more practical and useful for real-world network security. Based on these improvement the system block diagram is shown below.

4.1 System Block Diagram

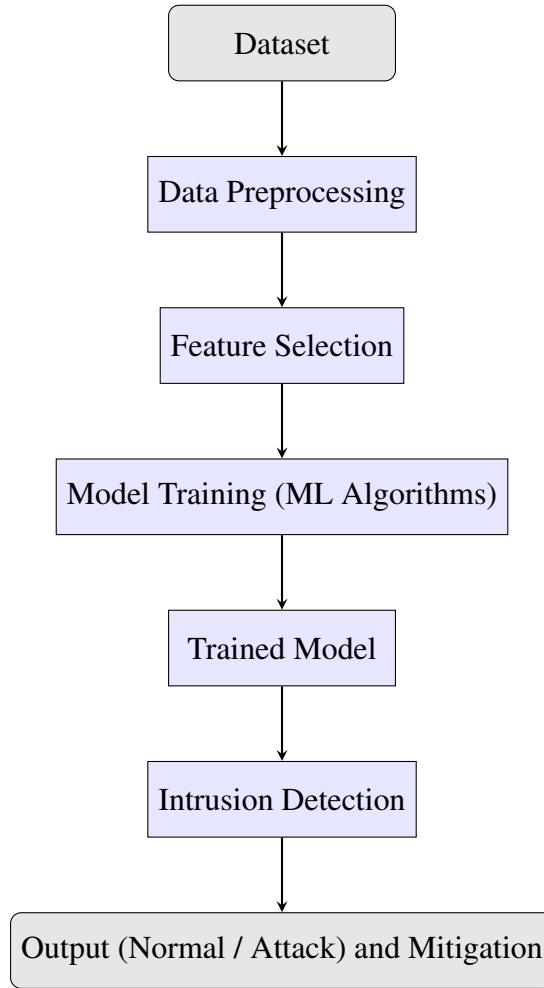


Figure 4.1: System Architecture

4.2 Block Descriptions

4.2.1 Dataset

The **CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017)** dataset is used for training and testing the models. The dataset contains realistic network traffic collected over five days and includes both benign traffic and various modern attack types, including:

- **DoS / DDoS** (Denial of Service / Distributed Denial of Service)
- **Brute Force** (FTP-Patator and SSH-Patator)
- **Web Attacks** (Brute Force, XSS, and SQL Injection)
- **Infiltration**
- **Botnet**
- **PortScan**
- **Heartbleed**

The dataset contains more than 80 network traffic features extracted using CICFlowMeter, providing detailed information about network flows and making it suitable for machine learning-based intrusion detection.

4.2.2 Data Preprocessing

The raw CICIDS2017 data is cleaned and transformed before model training. Missing and irrelevant values are handled, and the target labels are encoded into numerical form. Numerical features are scaled to ensure that features with different ranges can be processed effectively by the machine learning models. Class balancing techniques, including **SMOTETomek**, are applied to reduce the effect of class imbalance in the dataset.

4.2.3 Feature Selection

Relevant network traffic features are selected to improve the classification process and reduce unnecessary information. Removing irrelevant or redundant features helps re-

duce the dimensionality of the dataset, improve model efficiency, and reduce computational cost. Feature selection also helps minimize the risk of overfitting during model training.

4.2.4 Model Training

The processed and balanced dataset is used to train multiple machine learning models. The models implemented in the system are:

- **XGBoost**
- **Random Forest**
- **Naive Bayes**
- **LSTM**

Each model learns patterns from network traffic data to distinguish between benign and malicious activities. The models are trained and evaluated using the same processed dataset to enable a fair comparison of their performance.

4.2.5 Trained Model

After training, the models are saved and used to classify previously unseen network traffic. The trained models accept the required network traffic features as input and predict whether the traffic is benign or belongs to a particular attack category.

4.2.6 Intrusion Detection

The system analyzes unseen network traffic and classifies it into benign or attack categories. The supported attack categories include:

- **Benign**
- **DoS / DDoS**
- **Brute Force**
- **Web Attacks**

- **Botnet**
- **PortScan**
- **Infiltration**
- **Heartbleed**

The predicted class is generated by the trained machine learning model based on the characteristics of the input network traffic.

4.2.7 Output and Mitigation

The system produces a predicted traffic class indicating whether the input is benign or represents a specific type of network attack. For detected attacks, the system provides an appropriate **mitigation recommendation** based on the identified attack type. The mitigation component is intended to assist users in understanding the appropriate response to detected threats. The system's detection performance is evaluated using accuracy, precision, recall, F1-score, and false positive rate.

4.3 Data Flow Diagram

4.3.1 DFD Level 0

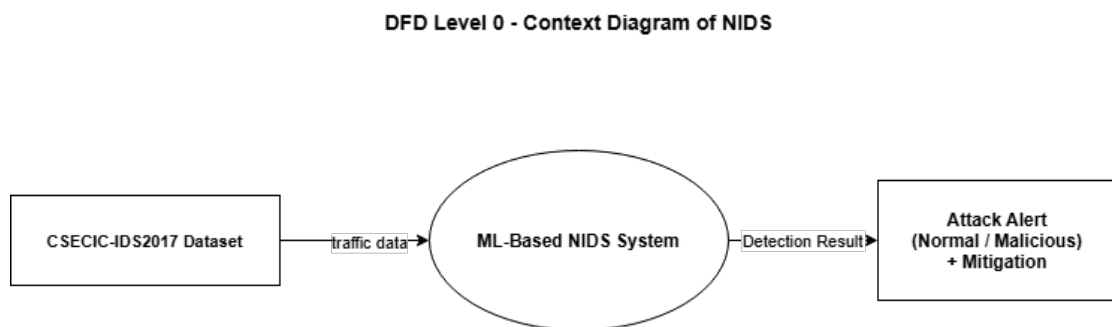


Figure 4.2: DFD Level 0-Context Diagram of NIDS

4.3.2 DFD Level 1

DFD Level 1 - Machine Learning Based NIDS System Flow

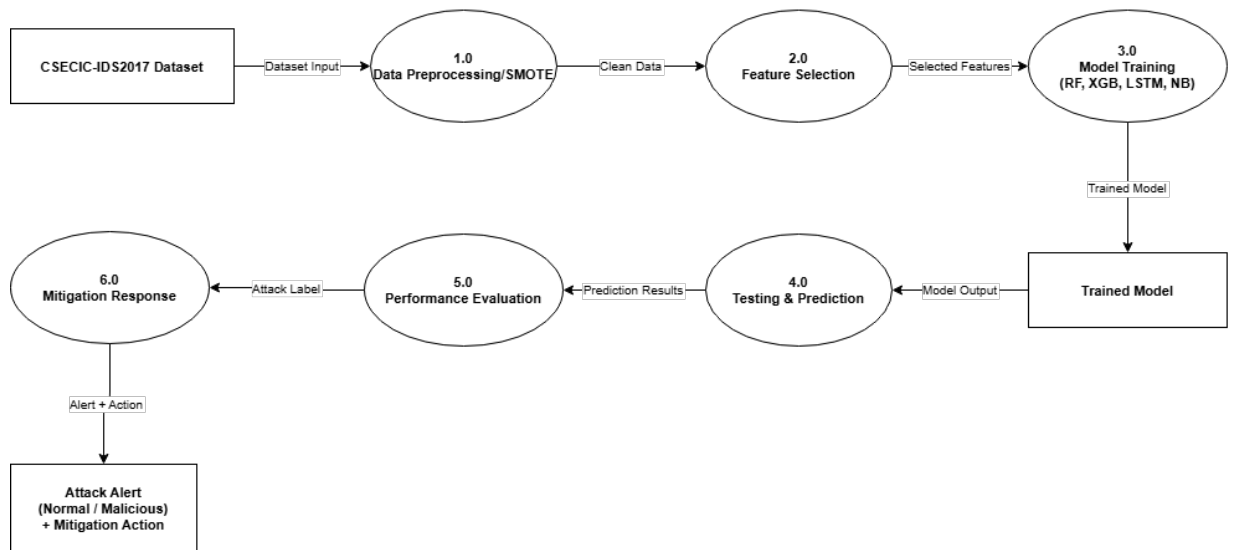


Figure 4.3: DFD Level 1-Machine Learning Based NIDS flow

5 METHODOLOGY

5.1 Overview

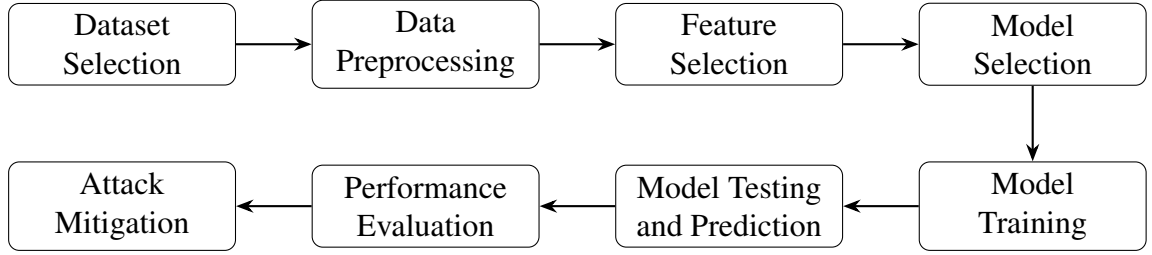


Figure 5.1: Methodology of the Proposed Network Intrusion Detection System

5.2 Detailed Description of Each Stage

5.2.1 Dataset Selection

The **CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017)** [11] dataset was selected for training and testing the proposed NIDS. It was created by the Canadian Institute for Cybersecurity and contains labeled network traffic captured in a realistic test environment over five days, from Monday to Friday. The dataset contains:

- More than **2.8 million** network traffic records
- More than **80 network traffic features** extracted using CICFlowMeter
- Various attack categories, including DoS, DDoS, Brute Force, Web Attacks, Botnet, PortScan, Infiltration, and Heartbleed
- Both benign and malicious network traffic

The CICIDS2017 dataset provides more recent and realistic network traffic patterns compared with older datasets such as KDD Cup 99 and NSL-KDD, making it suitable for evaluating machine learning-based intrusion detection systems.

5.2.2 Data Preprocessing

The raw network traffic data was preprocessed to improve data quality and prepare it for machine learning. The preprocessing process included removing invalid and irrelevant data, encoding the target labels, scaling numerical features, and addressing class imbalance.

Handling Missing and Inconsistent Data

The dataset was examined for missing, infinite, and inconsistent values. Invalid values were removed or replaced where necessary to ensure that the data could be processed correctly by the machine learning algorithms. This step helped improve the quality and reliability of the training data.

Feature Encoding and Scaling

The categorical target labels were converted into numerical representations using label encoding. This allowed the machine learning algorithms to process the different attack categories.

Numerical features were scaled so that features with different value ranges could be processed consistently. Min-Max scaling was used where applicable:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (5.1)$$

Irrelevant or redundant features were also removed where necessary to reduce the dimensionality of the dataset and improve computational efficiency.

Handling Class Imbalance Using SMOTETomek

The CICIDS2017 dataset contains significant class imbalance, where some attack categories contain substantially fewer samples than the majority classes. This imbalance can cause machine learning models to favor the majority class and perform poorly on minority attacks.

To address this issue, **SMOTETomek** was applied to the training data. The SMOTE-Tomek technique consists of two main steps: SMOTE for generating synthetic minority-class samples and Tomek Links for removing overlapping samples. The following equations describe the process.

The first equation represents the **SMOTE sample generation process**:

$$x_{\text{new}} = x_i + \lambda(x_{nn} - x_i), \quad 0 \leq \lambda \leq 1 \quad (5.2)$$

where x_i represents an existing minority-class sample, x_{nn} represents one of its nearest minority-class neighbors, and λ is a random value between 0 and 1. The new sample x_{new} is generated at a point between the two existing samples.

The second equation represents the **Euclidean distance** used to identify the nearest neighbors of a minority-class sample:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (5.3)$$

where x_i and x_j are two samples, x_{ik} and x_{jk} are their values for feature k , and n is the total number of features. A smaller distance indicates that two samples are more similar.

The third set of equations represents the **Tomek Link condition**:

$$d(x_i, x_j) = \min_{x_k \in C_j} d(x_i, x_k) \quad (5.4)$$

$$d(x_i, x_j) = \min_{x_k \in C_i} d(x_j, x_k) \quad (5.5)$$

These equations indicate that x_i and x_j form a Tomek Link when they belong to different classes and are the nearest neighbors of each other. Such samples are considered to be located near the class boundary and may be removed to reduce class overlap.

Tomek Links are then used to remove overlapping or borderline samples between different classes, improving class separation. Therefore, SMOTETomek combines over-sampling with data cleaning to produce a more balanced training dataset.

The process can be summarized as:

$$\text{Training Data} \rightarrow \text{SMOTE} \rightarrow \text{Tomek Links} \rightarrow \text{Balanced Training Data} \quad (5.6)$$

SMOTETomek was applied only to the training data, while the testing data remained

unchanged. This prevented data leakage and ensured that the final evaluation represented the model's performance on unseen data.

5.2.3 Feature Selection

Relevant network traffic features were selected to improve classification performance and reduce computational complexity. The features used in the dataset describe different characteristics of network flows, including:

Flow-Based Features: Flow duration, total forward and backward packets, total length of forward and backward packets, flow bytes/s, and flow packets/s.

Packet-Based Features: Packet length mean, packet length variance, packet length standard deviation, minimum packet length, and maximum packet length.

Flag-Based Features: FIN, SYN, RST, PSH, ACK, and URG flag counts.

Inter-Arrival Time Features: Flow IAT mean, flow IAT standard deviation, forward IAT mean, and backward IAT mean.

Activity Features: Active mean, active standard deviation, idle mean, and idle standard deviation.

Target Feature: The label column representing the traffic class, including BENIGN and the attack categories present in the processed dataset.

5.2.4 Model Training

The proposed Network Intrusion Detection System uses supervised machine learning algorithms to classify network traffic. The models implemented in the project are:

- **Random Forest**
- **XGBoost**
- **Naive Bayes**
- **LSTM**

After preprocessing, the dataset was divided into training and testing subsets. The train-

ing data was used to learn patterns associated with benign and malicious network traffic, while the testing data was reserved for evaluating the models on unseen samples.

An **80:20 split ratio** was used, where 80% of the data was used for training and 20% was used for testing. Stratified splitting was used to maintain the distribution of the target classes between the training and testing sets.

Table 5.1: Dataset Splitting Ratio

Dataset Type	Percentage
Training Data	80%
Testing Data	20%
Total Dataset	100%

The trained models were evaluated using performance metrics including accuracy, precision, recall, F1-score, and false positive rate. These metrics were used to compare the ability of each model to correctly identify different types of network traffic.

5.2.5 Model Selection

Multiple machine learning algorithms were trained and compared to identify the most effective model for the proposed NIDS.

Naives Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem. It assumes that all features are independent and calculates the probability of each class given the input features. Naive Bayes is based on Bayes' Theorem.

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)} \quad (5.7)$$

Where: $P(CX)$ = Posterior probability of class C , $P(XC)$ = Likelihood, $P(C)$ = Prior probability, $P(X)$ = Evidence,

For Multiple Features:

$$P(C|X_1, X_2, \dots, X_n) \propto P(C) \prod_{i=1}^n P(X_i|C) \quad (5.8)$$

Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees during training. Each tree independently predicts the class of a network traffic instance, and the final prediction is made using majority voting. Random Forest combines predictions from multiple decision trees.

In the proposed Network Intrusion Detection System, the Random Forest algorithm is used as one of the classification models. After preprocessing and feature selection of the CICIDS2017 dataset, the cleaned data is divided into training and testing sets. The training data is then used to build multiple decision trees using the bagging (bootstrap aggregation) technique.

Prediction:

$$Y' = \text{majorityvote}(T1(X), T2(X), \dots, Tn(X)) \quad (5.9)$$

Where:

$T_i(X)$ = prediction from the i -th decision tree, n = number of trees

For Classification

$$\hat{y} = \arg \max_c \sum_{i=1}^n I(T_i(x) = c) \quad (5.10)$$

Entropy:

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i) \quad (5.11)$$

Information Gain:

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) \quad (5.12)$$

XGBoost

XGBoost (Extreme Gradient Boosting) is an advanced ensemble technique that builds decision trees sequentially. Each new tree learns from the errors of previous trees, improving overall prediction performance

Final Prediction:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad (5.13)$$

Objective Function:

$$Obj = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (5.14)$$

Regularization Term

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (5.15)$$

LSTM (Long Short-Term Memory)

LSTM (Long Short-Term Memory) is a type of Recurrent Neural Network (RNN) specifically designed to learn long-term temporal dependencies in sequential data. In the context of Network Intrusion Detection Systems (NIDS), network traffic flows exhibit temporal patterns that LSTM can effectively learn and classify.

LSTM addresses the vanishing gradient problem in standard RNNs by introducing a cell state and three gating mechanisms: the forget gate, input gate, and output gate.

Forget Gate

The forget gate decides what information to discard from the cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5.16)$$

Input Gate

The input gate decides what new information to store in the cell state:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5.17)$$

Candidate Cell State

The candidate cell state creates new candidate values to be added to the cell state:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (5.18)$$

Cell State Update

The cell state is updated as follows:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (5.19)$$

Output Gate

The output gate determines what part of the cell state is output:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5.20)$$

Hidden State Output

$$h_t = o_t \odot \tanh(C_t) \quad (5.21)$$

Final Classification (Softmax)

For multi-class classification, the output probability is computed using the Softmax function:

$$P(y = c \mid x) = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}} \quad (5.22)$$

Where:

In the proposed NIDS, network traffic features from the CSE-CIC-IDS2017 dataset are reshaped into sequential time steps and fed into the LSTM network. The model learns temporal patterns of both normal and malicious network behavior across time windows and classifies each network flow accordingly.

Symbol	Description
x_t	Input at time step t
h_{t-1}	Previous hidden state
C_{t-1}	Previous cell state
W_f, W_i, W_C, W_o	Weight matrices
b_f, b_i, b_C, b_o	Bias vectors
σ	Sigmoid activation function
$\odot$	Element-wise (Hadamard) product
$\tanh$	Hyperbolic tangent activation function
C	Total number of output classes
z_c	Logit corresponding to class c

Table 5.2: Notation used in the LSTM model.

5.2.6 Model Testing and Prediction

The dataset was divided into training and testing sets, with 80% of the data used for training and 20% reserved for testing. The trained models were evaluated using unseen test data to determine their ability to classify network traffic. The model classifies traffic into the following categories:

- Normal (BENIGN)
- DoS Hulk
- DDoS
- PortScan
- DoS GoldenEye
- FTP-Patator
- DoS Slowloris
- DoS Slowhttptest
- SSH-Patator
- Bot
- Web Attack – Brute Force
- Web Attack – XSS

- Infiltration
- Web Attack – SQL Injection
- Heartbleed

The predicted class is then used by the attack mitigation component to determine the appropriate response to the detected network activity.

5.2.7 Performance Evaluation

The performance of the system will be evaluated using standard metrics such as:

- Accuracy

Accuracy is a performance metric that measures the overall correctness of the classification model. It calculates the ratio of correctly predicted instances (both normal and attack) to the total number of instances

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5.23)$$

- Precision

Precision measures how many of the instances predicted as attacks are actually correct. In a Network Intrusion Detection System, high precision means fewer false alarms.

$$Precision = \frac{TP}{TP + FP} \quad (5.24)$$

- Recall

Recall measures how many actual attack instances are correctly detected by the system. It is also known as the detection rate and is very important in cybersecurity applications

$$Recall = \frac{TP}{TP + FN} \quad (5.25)$$

- F1-score

F1-Score is the harmonic mean of Precision and Recall. It provides a balanced measure between detecting attacks correctly and minimizing false alarms.

It is especially useful when dealing with imbalanced datasets like NSL-KDD.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (5.26)$$

- **Confusion Matrix**

A Confusion Matrix is a performance evaluation tool used in classification problems, including Machine Learning-based Network Intrusion Detection Systems (NIDS). It summarizes the prediction results by comparing actual labels with predicted labels. It helps in understanding how well the model distinguishes between normal traffic and attack traffic.

$$\begin{bmatrix} TP & FN \\ FP & TN \end{bmatrix} \quad (5.27)$$

where TP=True Positive, TN=True Negative, FP=False Positive, FN=False Negative

5.2.8 Mitigation Methods

Once an intrusion is detected and classified, the system retrieves the corresponding mitigation strategy based on the predicted attack type. The mitigation database contains the severity, description, and recommended response for each detected class. The implemented mitigation methods are summarized in Table 5.3.

““

Table 5.3: Attack Types and Implemented Mitigation Methods

Attack Type	Mitigation Method
DDoS	Enable rate limiting, block attacker IPs, deploy DDoS protection, enable firewall filtering, and notify the administrator.
DoS GoldenEye	Rate limit HTTP traffic, deploy a Web Application Firewall (WAF), and block suspicious IPs.
DoS Hulk	Enable firewall rules, limit HTTP requests, and drop excessive packets.
DoS Slowhttptest	Reduce connection timeout, limit concurrent sessions, and use a reverse proxy.
DoS slowloris	Configure web server timeouts, limit open connections, and deploy a WAF.
FTP-Patator	Lock accounts after failed login attempts, enable multi-factor authentication (MFA), and enforce strong passwords.
SSH-Patator	Disable password authentication, use SSH keys, and block attacker IPs.
Web Attack – Brute Force	Enable CAPTCHA, lock accounts, and require strong passwords.
Web Attack – SQL Injection	Use prepared statements, validate user input, and deploy a Web Application Firewall.
Web Attack – XSS	Sanitize user input, escape HTML output, and enable Content Security Policy (CSP).
PortScan	Block the scanning IP, increase logging, and close unused ports.
Bot	Disconnect the infected host, run a malware scan, block the command-and-control (C2) server, and monitor outbound traffic.
Infiltration	Disconnect the infected machine, perform forensic investigation, and monitor lateral movement.
Heartbleed	Patch OpenSSL, replace affected certificates, and reset compromised credentials.
BENIGN	No mitigation action is required for normal network traffic.

5.3 Results

5.3.1 Experimental Setup

The proposed Network Intrusion Detection System (NIDS) was evaluated using four machine learning models: Naive Bayes, Random Forest, XGBoost, and Long Short-Term Memory (LSTM). The dataset was divided into training and testing sets using an 80:20 ratio. The models were initially trained using the preprocessed training data without applying SMOTETomek. This experiment was used as the baseline for evaluating the performance of the models.

The models were evaluated using accuracy, precision, recall, F1-score, training time, and prediction time. Since the dataset contains multiple attack classes with significant class imbalance, class-wise performance was also considered, particularly for minority attack classes.

After the initial experiment, SMOTETomek was applied to the training data to address class imbalance. Naive Bayes, Random Forest, and XGBoost were retrained using the balanced training data. SMOTETomek was not applied to the LSTM model because the sequence-based training approach used for LSTM was different from the preprocessing approach used for the other models.

5.3.2 Results Before Applying SMOTETomek

The initial experiment was conducted without applying SMOTETomek. The classification reports for Naive Bayes, Random Forest, XGBoost, and LSTM were generated using the same test dataset. The results were used to establish a baseline for comparing the effect of SMOTETomek.

Naive Bayes

Naive Bayes achieved an accuracy of 72.00%, with a weighted precision of 98.00% and an F1-score of 82.00%. Although the model achieved relatively high weighted precision, its overall classification performance was considerably lower than that of the other models.

The classification report for Naive Bayes is shown in Figure 5.2.

1		precision	recall	f1-score	support
2	BENIGN	0.999505	0.679847	0.809253	419012
3	Bot	0.003926	1	0.00782	390
4	DDoS	0.94057	0.930321	0.935417	25603
5	DoS Golde	0.175253	0.917842	0.29431	2057
6	DoS Hulk	0.88829	0.929995	0.908664	34569
7	DoS Slowh	0.145203	0.693117	0.240106	1046
8	DoS slowl	0.047802	0.636026	0.088921	1077
9	FTP-Patate	0.684638	0.995784	0.811405	1186
10	Heartbleec	0	0	0	2
11	Infiltration	0.006972	1	0.013848	7
12	PortScan	0.979301	0.978113	0.978707	18139
13	SSH-Patate	0.363018	0.911491	0.519239	644
14	Web Attac	0.008226	0.12585	0.015442	294
15	Web Attac	0.008791	1	0.017429	4
16	Web Attac	0.155327	0.930769	0.266227	130
17	accuracy	0.722388	0.722388	0.722388	0.722388
18	macro avg	0.360455	0.781944	0.393786	504160
19	weighted a	0.977846	0.722388	0.822144	504160

Figure 5.2: Naive Bayes classification report before SMOTETomek

Random Forest

Random Forest achieved an accuracy of 99.83%, with a precision of 99.83%, recall of 99.86%, and F1-score of 99.82%. The model demonstrated strong performance across the majority of the attack classes.

The classification report for Random Forest is shown in Figure 5.3.

1		precision	recall	f1-score	support
2	BENIGN	0.999003	0.999301	0.999152	419012
3	Bot	0.907895	0.530769	0.669903	390
4	DDoS	0.999804	0.998477	0.99914	25603
5	DoS Golde	0.992165	0.98493	0.988534	2057
6	DoS Hulk	0.998204	0.996992	0.997598	34569
7	DoS Slowh	0.987619	0.991396	0.989504	1046
8	DoS slowl	0.992509	0.984215	0.988345	1077
9	FTP-Patate	1	0.996627	0.998311	1186
10	Heartbleec	0	0	0	2
11	Infiltration	1	0.142857	0.25	7
12	PortScan	0.989193	0.999173	0.994158	18139
13	SSH-Patate	1	0.97205	0.985827	644
14	Web Attac	0.736025	0.806122	0.769481	294
15	Web Attac	0	0	0	4
16	Web Attac	0.505882	0.330769	0.4	130
17	accuracy	0.998276	0.998276	0.998276	0.998276
18	macro avg	0.80722	0.715579	0.73533	504160
19	weighted a	0.998211	0.998276	0.998194	504160

Figure 5.3: Random Forest classification report before SMOTETomek

XGBoost

XGBoost achieved the highest overall performance among the evaluated models. It obtained an accuracy of 99.87%, precision of 99.86%, recall of 99.87%, and F1-score of 99.87%.

The classification report for XGBoost is shown in Figure 5.4.

1		precision	recall	f1-score	support
2	0	0.999549	0.999267	0.999408	419012
3	1	0.855114	0.771795	0.811321	390
4	2	0.999961	0.999453	0.999707	25603
5	3	0.996585	0.993194	0.994887	2057
6	4	0.998467	0.998872	0.99867	34569
7	5	0.983902	0.993308	0.988582	1046
8	6	0.993464	0.987929	0.990689	1077
9	7	1	0.99747	0.998734	1186
10	8	0	0	0	2
11	9	1	0.285714	0.444444	7
12	10	0.989082	0.998897	0.993966	18139
13	11	1	0.996894	0.998445	644
14	12	0.742138	0.802721	0.771242	294
15	13	0	0	0	4
16	14	0.45	0.346154	0.391304	130
17	accuracy	0.998687	0.998687	0.998687	0.998687
18	macro avg	0.800551	0.744778	0.75876	504160
19	weighted avg	0.998648	0.998687	0.998659	504160

Figure 5.4: XGBoost classification report before SMOTETomek

LSTM

The LSTM model achieved an accuracy of 98.62%, with a precision of 98.55%, recall of 98.62%, and F1-score of 98.55%. Although the model provided strong classification performance, its prediction time was considerably higher than the tree-based models. Therefore, it provided lower practical efficiency for the proposed NIDS compared with XGBoost.

The classification report for LSTM is shown in Figure 5.7.

1		precision	recall	f1-score	support
2	0	0.989975	0.993599	0.991784	419012
3	1	0.971429	0.348718	0.513208	390
4	2	0.998984	0.998203	0.998593	25603
5	3	0.966475	0.98104	0.973703	2057
6	4	0.974372	0.989846	0.982048	34569
7	5	0.888508	0.99044	0.936709	1046
8	6	0.965974	0.948932	0.957377	1077
9	7	0.972546	0.985666	0.979062	1186
10	8	0	0	0	2
11	9	0	0	0	7
12	10	0.907798	0.833728	0.869188	18139
13	11	0.998276	0.899068	0.946078	644
14	12	0.6	0.040816	0.076433	294
15	13	0	0	0	4
16	14	0	0	0	130
17	accuracy	0.986195	0.986195	0.986195	0.986195
18	macro avg	0.682289	0.600671	0.614946	504160
19	weighted avg	0.985495	0.986195	0.985516	504160

Figure 5.5: LSTM classification report

The accuracy graph for LSTM is shown in Figure 5.7.

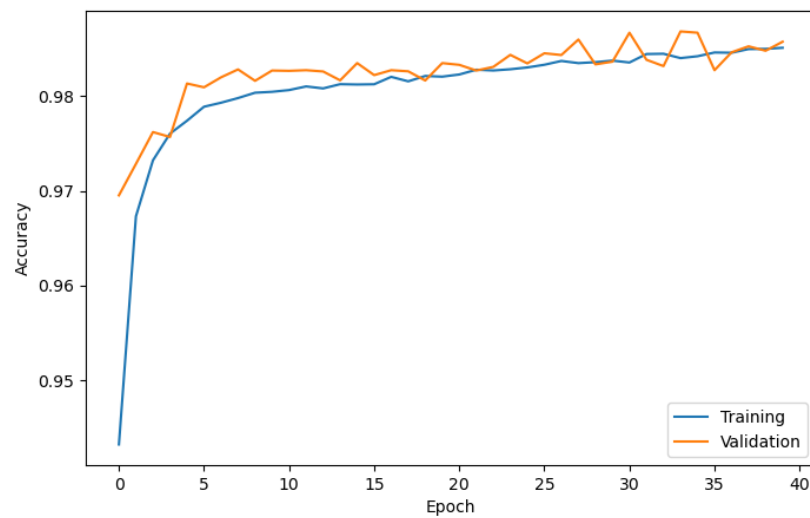


Figure 5.6: LSTM Accuracy Graph

The Loss graph for LSTM is shown in Figure 5.7.

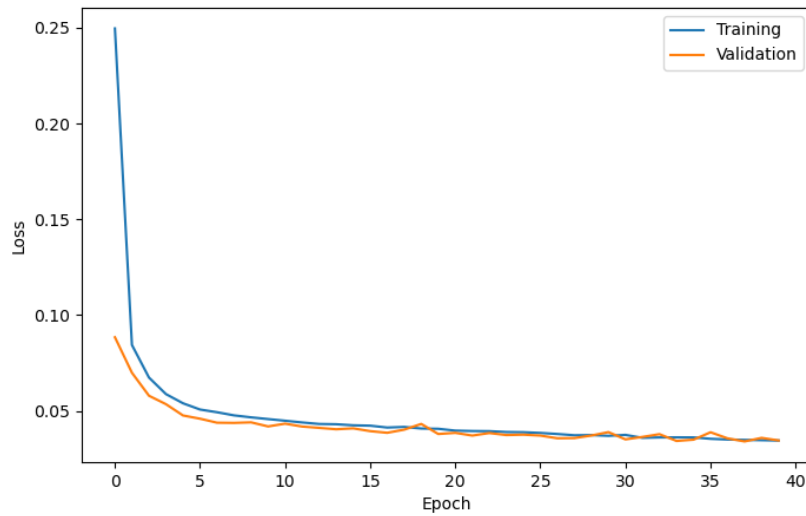


Figure 5.7: LSTM Loss Graph

5.3.3 Comparison of Machine Learning Models

The overall performance of the four models is presented in Table 5.4.

The performance of the models before applying SMOTETomek is summarized in Table 5.5.

Table 5.4: Model Performance before SMOTETomek

Model	Accuracy	Precision	Recall	F1	Train (s)	Predict (s)
Naive Bayes	0.720000	0.980000	0.720000	0.820000	0.80	0.20
Random Forest	0.998276	0.998211	0.998276	0.998194	86.37	11.57
XGBoost	0.998687	0.998648	0.998687	0.998659	180.23	5.00
LSTM	0.986195	0.985495	0.986195	0.985516	435.63	38.73

The comparative bar graphs before applying SMOTETomek are shown as follows:

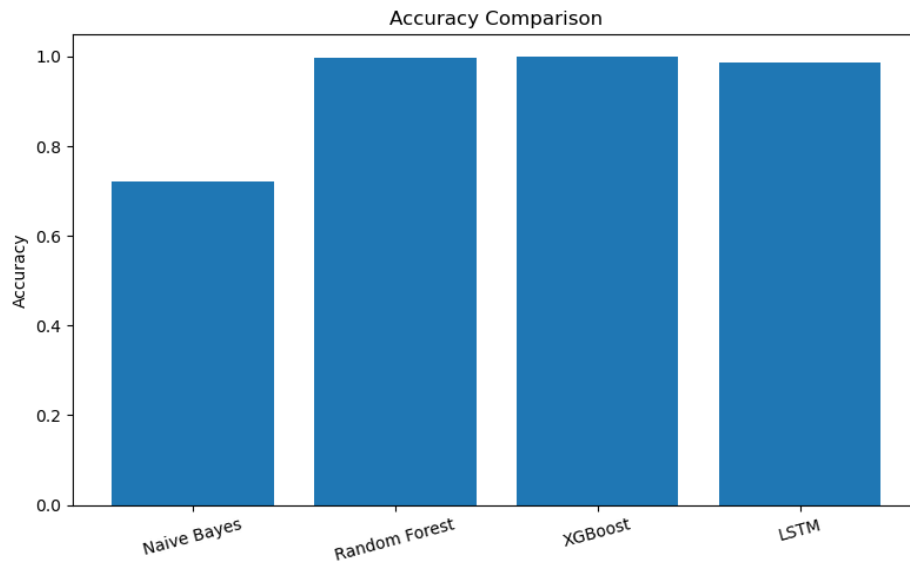


Figure 5.8: Accuracy comparison of machine learning models before SMOTETomek

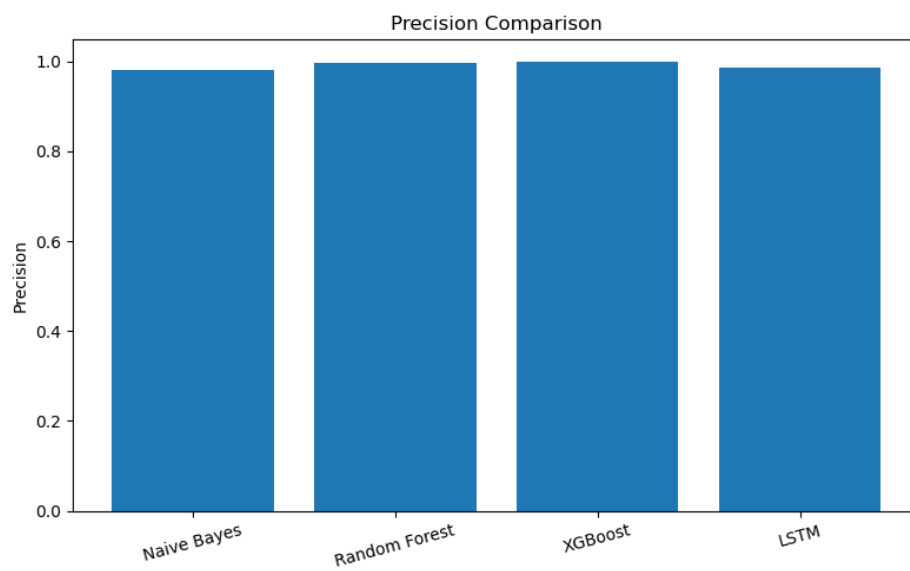


Figure 5.9: precision comparison of machine learning models before SMOTETomek

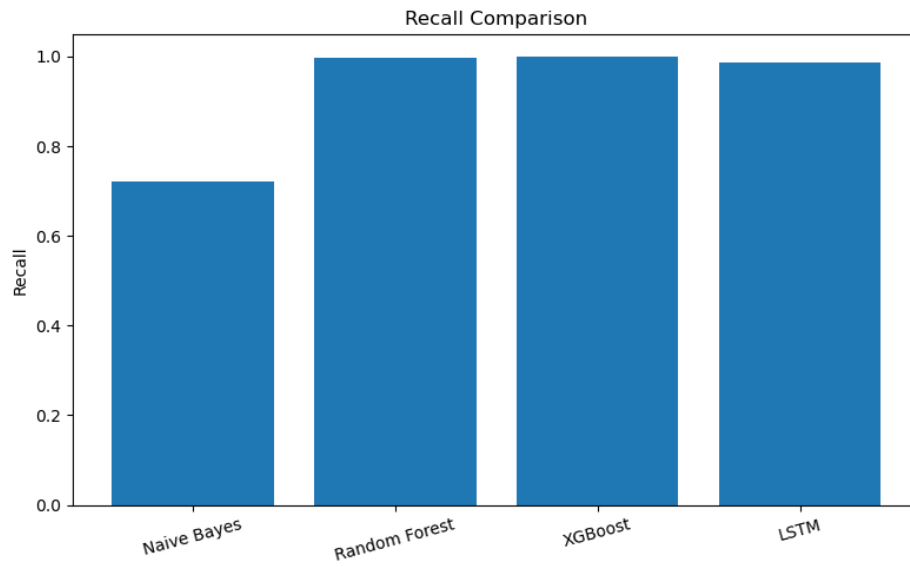


Figure 5.10: Recall comparison of machine learning models before SMOTETomek

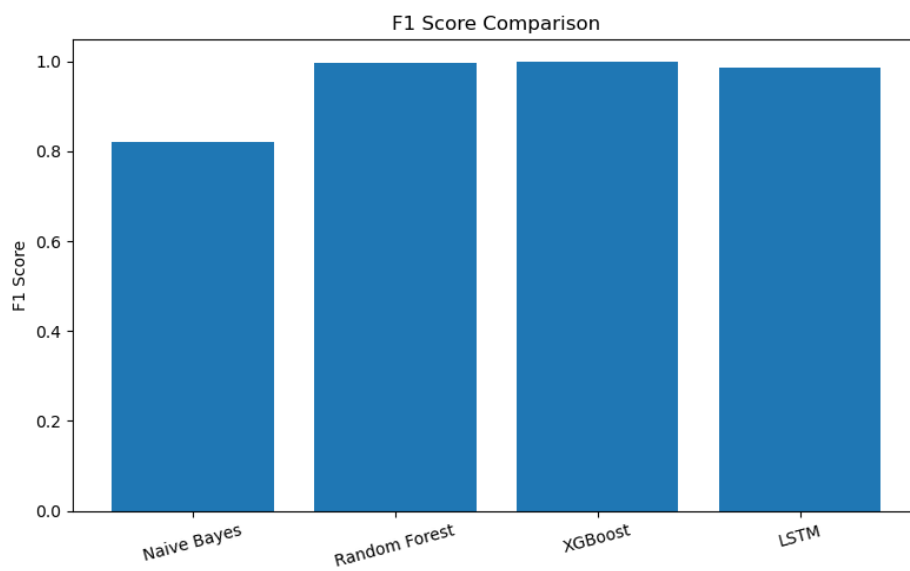


Figure 5.11: F1 comparison of machine learning models before SMOTETomek

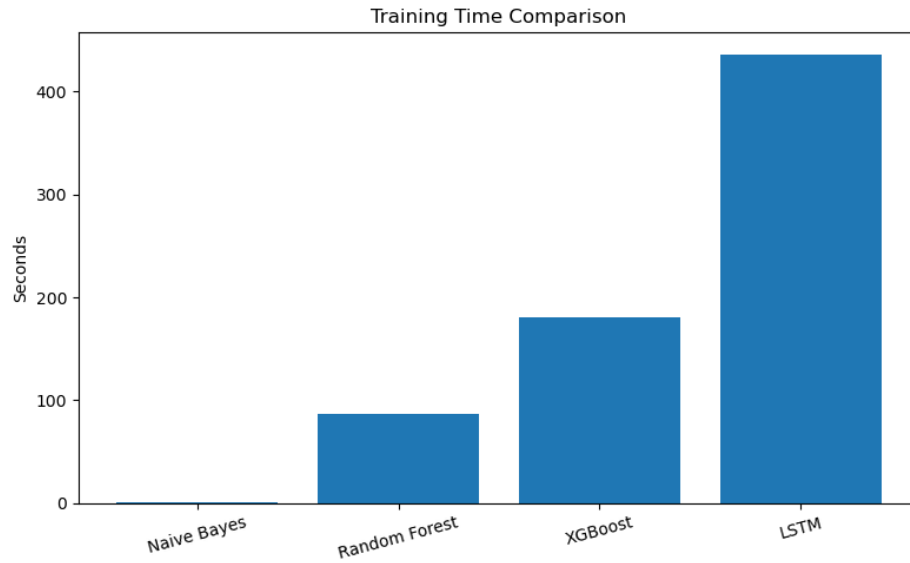


Figure 5.12: Training time comparison of machine learning models before SMOTE-Tomek

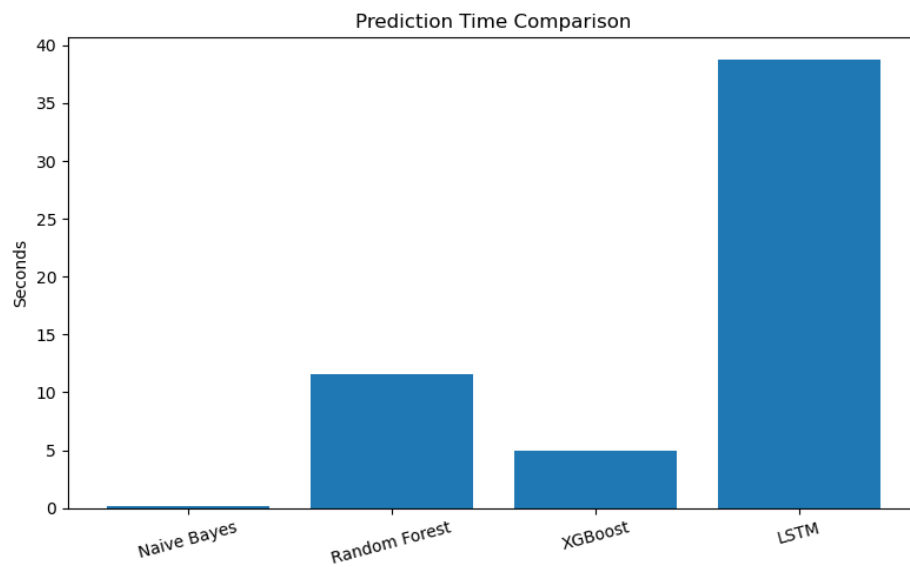


Figure 5.13: Prediction time comparison of machine learning models before SMOTE-Tomek

The comparative results show that XGBoost achieved the highest overall accuracy, precision, recall, and F1-score. Random Forest performed very closely to XGBoost, while LSTM also achieved high classification performance. Naive Bayes, however, produced considerably lower overall performance.

5.3.4 Results After Applying SMOTETomek

The dataset contained significant class imbalance, with some attack classes having substantially fewer samples than the majority classes. SMOTETomek was therefore applied to the training data to improve the representation of minority classes and remove potentially ambiguous borderline samples.

After applying SMOTETomek, Naive Bayes, Random Forest, and XGBoost were re-trained using the balanced training data. The same test dataset was retained for evaluating the models.

Naive Bayes with SMOTETomek

After applying SMOTETomek, Naive Bayes achieved an accuracy of 71.42%, precision of 97.67%, recall of 72.00%, and F1-score of 81.57%. The classification report is shown in Figure 5.14.

1		precision	recall	f1-score	support
2	BENIGN	0.999569	0.669558	0.80194	419012
3	Bot	0.00373	1	0.007432	390
4	DDoS	0.9446	0.976292	0.960184	25603
5	DoS Golde	0.147013	0.874575	0.251714	2057
6	DoS Hulk	0.876787	0.929417	0.902335	34569
7	DoS Slowh	0.155153	0.66348	0.251495	1046
8	DoS slowl	0.051865	0.641597	0.095972	1077
9	FTP-Patate	0.685432	0.995784	0.811963	1186
10	Heartbleec	1	0.5	0.666667	2
11	Infiltration	0.005627	1	0.011191	7
12	PortScan	0.962558	0.932576	0.94733	18139
13	SSH-Patate	0.362848	0.909938	0.518814	644
14	Web Attac	0.005019	0.040816	0.008939	294
15	Web Attac	0.004884	1	0.009721	4
16	Web Attac	0.296651	0.953846	0.452555	130
17	accuracy	0.714224	0.714224	0.714224	0.714224
18	macro avg	0.433449	0.805859	0.44655	504160
19	weighted a	0.976666	0.714224	0.815673	504160

Figure 5.14: Naive Bayes classification report after SMOTETomek

Random Forest with SMOTETomek

Random Forest achieved an accuracy of 99.86%, precision of 99.85%, recall of 99.86%, and F1-score of 99.85% after applying SMOTETomek. The results remained very high.

The classification report is shown in Figure 5.15.

1		precision	recall	f1-score	support
2	BENIGN	0.999294	0.999358	0.999326	419012
3	Bot	0.896057	0.641026	0.747384	390
4	DDoS	0.999961	0.999648	0.999805	25603
5	DoS Golde	0.995601	0.990277	0.992932	2057
6	DoS Hulk	0.998928	0.997223	0.998075	34569
7	DoS Slowh	0.992381	0.996176	0.994275	1046
8	DoS slowl	0.994398	0.988858	0.99162	1077
9	FTP-Patato	1	0.99747	0.998734	1186
10	Heartbleec	1	0.5	0.666667	2
11	Infiltration	0.875	1	0.933333	7
12	PortScan	0.9893	0.999063	0.994158	18139
13	SSH-Patato	1	0.98913	0.994536	644
14	Web Attac	0.748322	0.758503	0.753378	294
15	Web Attac	0.125	0.25	0.166667	4
16	Web Attac	0.444444	0.4	0.421053	130
17	accuracy	0.998552	0.998552	0.998552	0.998552
18	macro avg	0.870579	0.833782	0.843463	504160
19	weighted a	0.998528	0.998552	0.998522	504160

Figure 5.15: Random Forest classification report after SMOTETomek

XGBoost with SMOTETomek

After applying SMOTETomek, XGBoost achieved an accuracy of 99.88%, precision of 99.88%, recall of 99.88%, and F1-score of 99.88%. Although these values were slightly lower than the baseline XGBoost results, the model continued to demonstrate very strong classification performance.

The classification report is shown in Figure 5.16.

1		precision	recall	f1-score	support
2	0	0.999644	0.999317	0.999481	419012
3	1	0.888235	0.774359	0.827397	390
4	2	0.999961	1	0.99998	25603
5	3	0.996589	0.994166	0.995376	2057
6	4	0.998641	0.998756	0.998698	34569
7	5	0.984848	0.994264	0.989534	1046
8	6	0.995331	0.989786	0.992551	1077
9	7	1	0.998314	0.999156	1186
10	8	1	1	1	2
11	9	1	0.857143	0.923077	7
12	10	0.989247	0.999173	0.994185	18139
13	11	1	1	1	644
14	12	0.754266	0.751701	0.752981	294
15	13	0.8	1	0.888889	4
16	14	0.432836	0.446154	0.439394	130
17	accuracy	0.998792	0.998792	0.998792	0.998792
18	macro avg	0.92264	0.920209	0.920047	504160
19	weighted avg	0.998789	0.998792	0.998787	504160

Figure 5.16: XGBoost classification report after SMOTETomek

5.3.5 Comparison After Applying SMOTETomek

Table 5.5: Performance Comparison of Models after SMOTETomek

Model	Accuracy	Precision	Recall	F1	Train (s)	Predict (s)
Naive Bayes	0.714224	0.976666	0.714224	0.815673	5.57	10.97
Random Forest	0.998552	0.998528	0.998552	0.998522	273.66	7.89
XGBoost	0.998792	0.998789	0.998792	0.998787	443.79	4.39
LSTM	0.986195	0.985495	0.986195	0.985516	435.63	38.73

The comparative bar graphs are presented as followed:

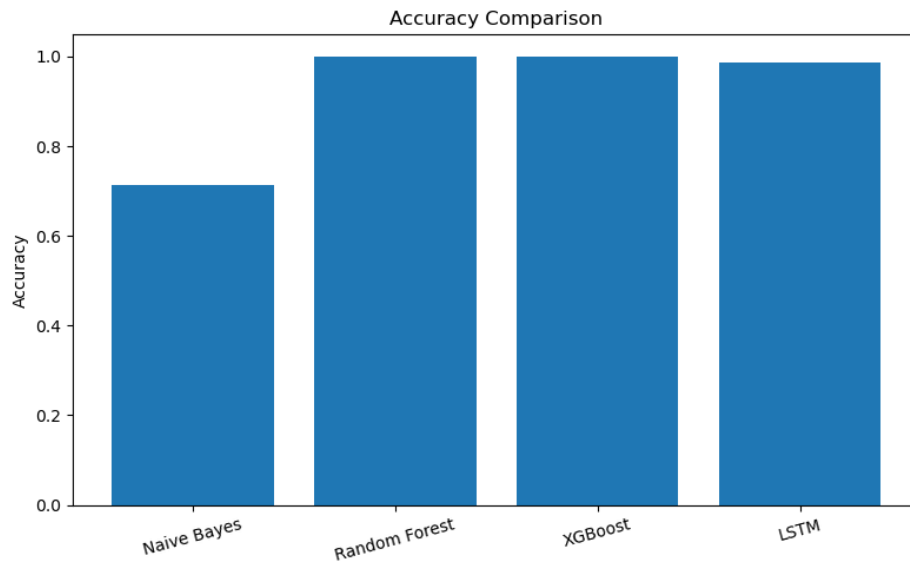


Figure 5.17: Accuracy comparison of machine learning models after SMOTETomek

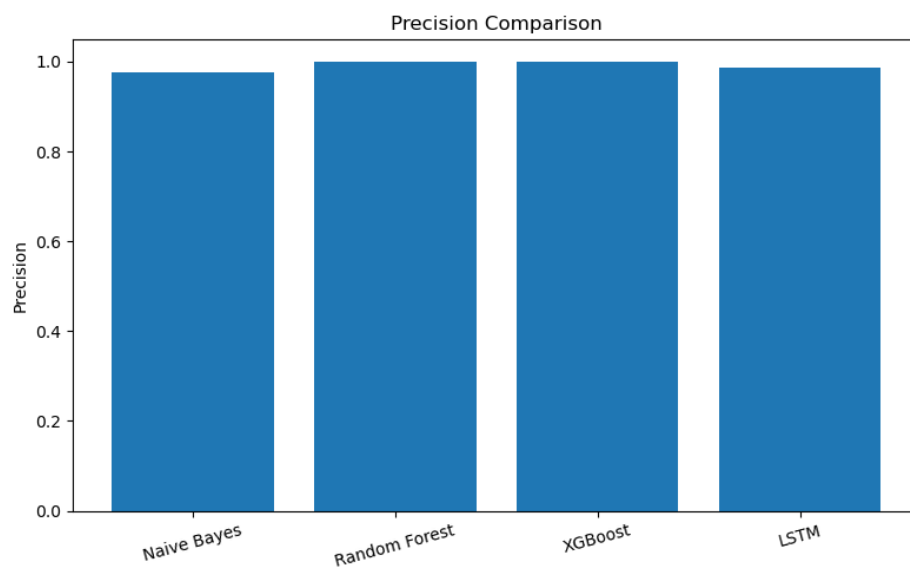


Figure 5.18: precision comparison of machine learning models after SMOTETomek

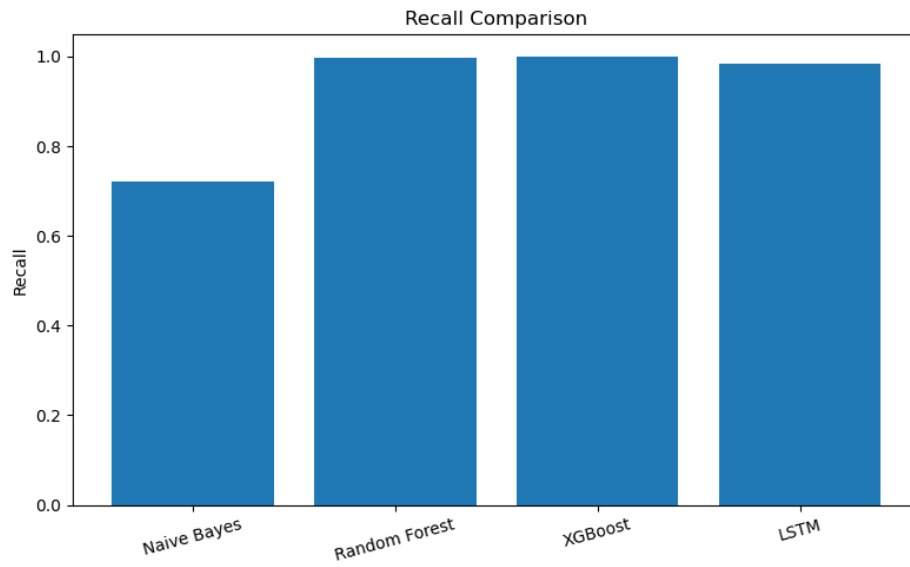


Figure 5.19: Recall comparison of machine learning models after SMOTETomek

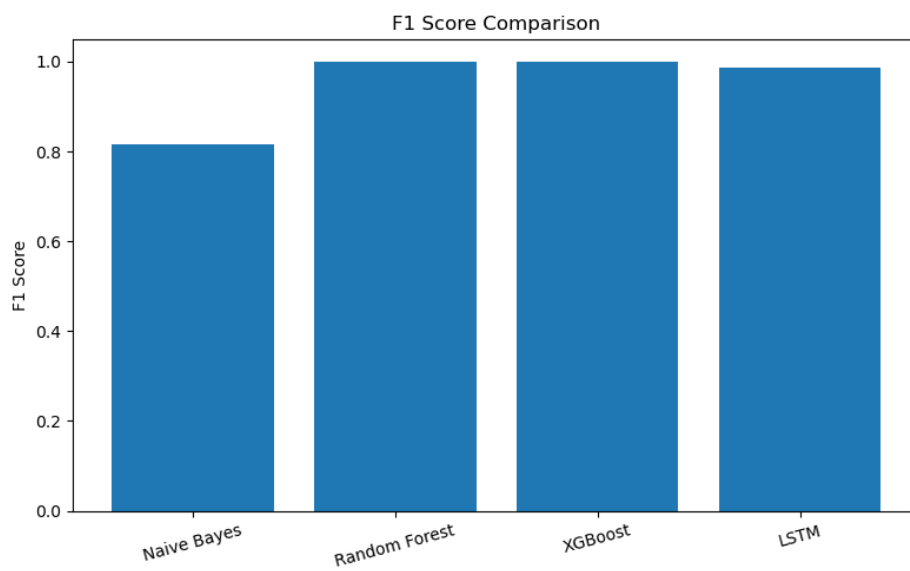


Figure 5.20: F1 comparison of machine learning models after SMOTETomek

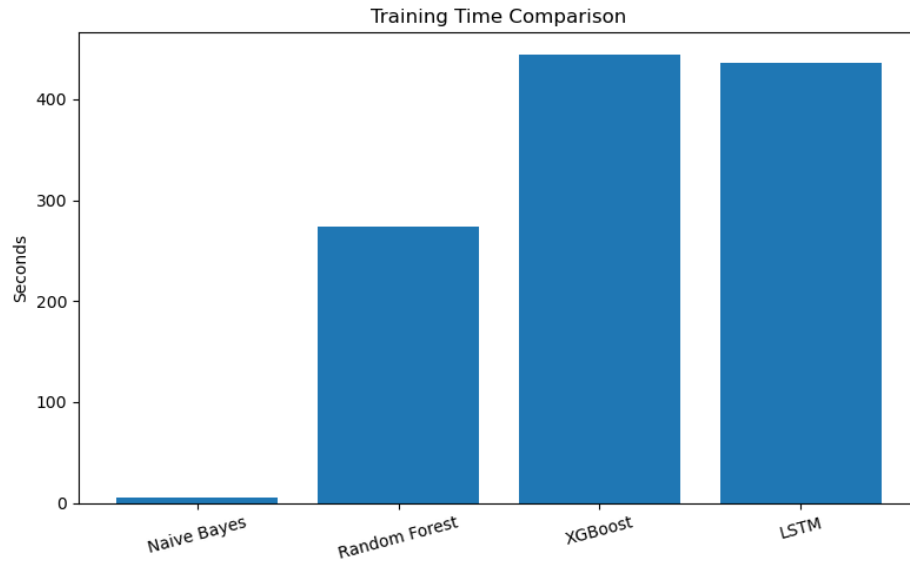


Figure 5.21: Training time comparison of machine learning models after SMOTE-Tomek

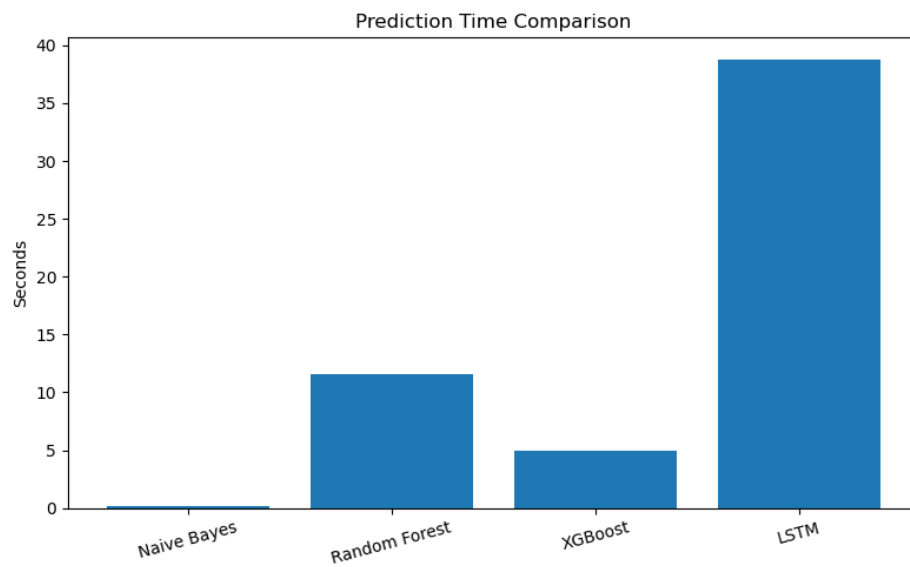


Figure 5.22: Prediction time comparison of machine learning models after SMOTE-Tomek

5.3.6 Effect of SMOTETomek on Model Performance

The results before and after applying SMOTETomek were compared to determine the effect of the balancing technique.

Table 5.6: Comparison Before and After SMOTETomek

Model	Method	Accuracy	Precision	Recall	F1-Score
Naive Bayes	Before	0.720000	0.980000	0.720000	0.820000
Naive Bayes	SMOTETomek	0.714224	0.976666	0.714224	0.815673
Random Forest	Before	0.998276	0.998211	0.998276	0.998194
Random Forest	SMOTETomek	0.998552	0.998528	0.998552	0.998522
XGBoost	Before	0.998687	0.998648	0.998687	0.998659
XGBoost	SMOTETomek	0.998792	0.998789	0.998792	0.998787

5.3.7 Selection of XGBoost

After comparing the results before and after applying SMOTETomek, XGBoost was selected as the primary model for the proposed NIDS. Its accuracy, precision, and F1-score improved from 99.87

5.3.8 Minority Attack Detection

The highly imbalanced nature of the dataset means that overall accuracy can be dominated by the majority classes. Therefore, the class-wise precision, recall, and F1-score were examined to determine how effectively the models detected minority attack categories.

The comparison of minority attack detection before and after applying SMOTETomek is presented in Figure 5.25.

Metric	Before SMOTE	After SMOTE
Accuracy	78.67%	91.47%
Macro Precision	83%	93%
Macro Recall	79%	91%
Macro F1-score	77%	92%
Weighted Precision	83%	93%
Weighted Recall	79%	91%
Weighted F1-score	77%	92%

Figure 5.23: Comparison of minority attack detection before and after SMOTETomek

Class	Precision	Recall	F1	Support
BENIGN	0.31	1.00	0.48	5
Bot	1.00	1.00	1.00	5
DDoS	1.00	1.00	1.00	5
DoS GoldenEye	1.00	1.00	1.00	5
DoS Hulk	1.00	1.00	1.00	5
DoS Slowhttptest	1.00	1.00	1.00	5
DoS slowloris	1.00	1.00	1.00	5
FTP-Patator	1.00	1.00	1.00	5
Heartbleed	0.00	0.00	0.00	5
Infiltration	1.00	0.40	0.57	5
PortScan	1.00	1.00	1.00	5
SSH-Patator	1.00	1.00	1.00	5
Web Attack – Brute Force	0.50	0.80	0.62	5
Web Attack – SQL Injection	1.00	0.20	0.33	5
Web Attack – XSS	0.67	0.40	0.50	5

Figure 5.24: Per-Class detection before SmoteTomek

Class	Precision	Recall	F1
BENIGN	0.67	1.00	0.80
Bot	1.00	0.87	0.93
DDoS	1.00	1.00	1.00
DoS GoldenEye	1.00	1.00	1.00
DoS Hulk	0.97	1.00	0.98
DoS Slowhttptest	1.00	1.00	1.00
DoS slowloris	1.00	1.00	1.00
FTP-Patator	1.00	1.00	1.00
Heartbleed	1.00	1.00	1.00
Infiltration	1.00	0.80	0.89
PortScan	1.00	1.00	1.00
SSH-Patator	1.00	1.00	1.00
Web Attack – Brute Force	0.65	0.73	0.69
Web Attack – SQL Injection	1.00	0.71	0.83
Web Attack – XSS	0.69	0.60	0.64

Figure 5.25: Per-Class detection after SmoteTomek

The minority-class results provide additional information about the effectiveness of SMOTETomek that cannot be obtained from overall accuracy alone. In particular, rare attack classes such as Heartbleed, Infiltration, and Web Attack categories require class-wise evaluation because they contain substantially fewer samples than the majority classes.

The observed changes in minority-class precision, recall, and F1-score were therefore

considered when evaluating the effect of SMOTETomek. The results demonstrate that the models were capable of identifying several minority attack categories despite their limited representation in the original dataset.

5.3.9 Confusion Matrix Analysis

Confusion matrices were used to provide a visual representation of the classification performance of each model. They show the number of correctly and incorrectly classified samples for each attack category and allow misclassification between different classes to be examined.

The confusion matrices after applying SMOTETomek are presented below.

Confusion Matrices After SMOTETomek

[[280553 100626 1455 9960 4496 3729 12219 538 0 1237									
658 1029 2317 144 51]									
[	0	390	0	0	0	0	0	0	0
0 0 0 0 0]									
[	17	556	24996	11	17	4	0	0	0
0 0 2 0 0]									
[	0	17	0	1799	1	4	0	0	0
0 0 0 236 0]									
[	49	1764	0	143	32129	0	128	0	0
0 0 9 347 0]									
[	0	0	0	5	1	694	285	0	0
0 0 30 31 0]									
[	0	5	0	311	0	41	691	0	0
0 0 2 27 0]									
[	2	2	0	0	0	0	0	1181	0
0 0 0 0 1]									
[	1	0	0	0	0	0	0	0	1
0 0 0 0 0]									
[	0	0	0	0	0	0	0	0	7
0 0 0 0 0]									
[	51	1131	11	7	0	1	0	0	0
16916 0 19 1 2]									
[	1	53	0	0	0	0	0	4	0
0 586 0 0 0]									
[	0	13	0	0	0	0	0	0	0
0 0 12 29 240]									
[	0	0	0	0	0	0	0	0	0
0 0 0 4 0]									
[	0	5	0	1	0	0	0	0	0
0 0 0 0 124]]									

Figure 5.26: Confusion matrices of the Naives Bayes after SMOTETomek

```

[[418743    29     1     4    28     7     2     0     0     1
   195      0     1     1     0]
 [ 140    250     0     0     0     0     0     0     0     0
   0      0     0     0     0]
 [   8      0  25594     0     1     0     0     0     0     0
   0      0     0     0     0]
 [  16      0     0  2037     4     0     0     0     0     0
   0      0     0     0     0]
 [  91      0     0     4  34473     0     0     0     0     0
   1      0     0     0     0]
 [   0      0     0     0     0  1042     4     0     0     0
   0      0     0     0     0]
 [  10      0     0     0     0     1  1065     0     0     0
   0      0     0     1     0]
 [   2      0     0     0     0     0     0  1183     0     0
   0      0     0     0     1]
 [   1      0     0     0     0     0     0     0     1     0
   0      0     0     0     0]
 [   0      0     0     0     0     0     0     0     0     7
   0      0     0     0     0]
 [  10      0     0     0     4     0     0     0     0     0
  18122     0     1     0     2]
 [   7      0     0     0     0     0     0     0     0     0
   0    637     0     0     0]
 [   4      0     0     0     0     0     0     0     0     0
   0      0    223     5    62]
 [   1      0     0     1     0     0     0     0     0     0
   0      0     1     1     0]
 [   6      0     0     0     0     0     0     0     0     0
   0      0    72     0    52]]

```

Figure 5.27: Confusion matrices of the Random Forest after SMOTETomek

```

[[418726    38     1     2    39     7     1     0     0     0
   196      0     1     0    1]
 [   88    302     0     0     0     0     0     0     0     0
   0      0     0     0     0]
 [   0      0  25603     0     0     0     0     0     0     0
   0      0     0     0     0]
 [   5      0     0  2045     4     2     0     0     0     0
   0      0     0     0     1]
 [  37      0     0     5  34526     0     0     0     0     0
   1      0     0     0     0]
 [   1      0     0     0     0  1040     4     0     0     0
   0      0     0     0     1]
 [   3      0     0     0     0     7  1066     0     0     0
   0      0     0     1     0]
 [   1      0     0     0     0     0     0  1184     0     0
   0      0     0     0     1]
 [   0      0     0     0     0     0     0     0     2     0
   0      0     0     0     0]
 [   1      0     0     0     0     0     0     0     0     6
   0      0     0     0     0]
 [   7      0     0     0     4     0     0     0     0     0
  18124     0     2     0     2]
 [   0      0     0     0     0     0     0     0     0     0
   0    644     0     0     0]
 [   3      0     0     0     0     0     0     0     0     0
   0      0    221     0    70]
 [   0      0     0     0     0     0     0     0     0     0
   0      0     0     4     0]
 [   3      0     0     0     0     0     0     0     0     0
   0      0    69     0    58]]

```

Figure 5.28: Confusion matrices of the XGBoost after SMOTETomek

[	416330	4	26	64	887	124	27	14	0	0
	1532	1	3	0	0]					
[	254	136	0	0	0	0	0	0	0	0
	0	0	0	0	0]					
[	37	0	25557	5	4	0	0	0	0	0
	0	0	0	0	0]					
[	39	0	0	2018	0	0	0	0	0	0
	0	0	0	0	0]					
[	351	0	0	0	34218	0	0	0	0	0
	0	0	0	0	0]					
[	6	0	0	0	0	1036	4	0	0	0
	0	0	0	0	0]					
[	32	0	0	0	0	6	1022	17	0	0
	0	0	0	0	0]					
[	17	0	0	0	0	0	0	1169	0	0
	0	0	0	0	0]					
[	2	0	0	0	0	0	0	0	0	0
	0	0	0	0	0]					
[	5	0	0	0	0	0	0	0	0	0
	0	0	2	0	0]					
[	3002	0	0	0	0	9	5	0	0	0
	15123	0	0	0	0]					
[	59	0	0	0	0	0	0	2	0	0
	4	579	0	0	0]					
[	282	0	0	0	0	0	0	0	0	0
	0	0	12	0	0]					
[	3	0	0	1	0	0	0	0	0	0
	0	0	0	0	0]					
[	127	0	0	0	0	0	0	0	0	0
	0	0	3	0	0]]					

Figure 5.29: Confusion matrices of the LSTM after SMOTETomek

5.3.10 Final Model Performance

Based on the comparative experiments, XGBoost was selected as the final model for the proposed NIDS. Before applying SMOTETomek, it achieved 99.8687% accuracy, 99.8648% precision, and 99.8659% F1-score. After applying SMOTETomek, these improved slightly to 99.8792%, 99.8789%, and 99.8787%, respectively. This improvement shows that SMOTETomek helped enhance the model's performance by addressing class imbalance. Therefore, the XGBoost model trained on the SMOTETomek-balanced dataset was selected and integrated with the prediction and mitigation components.

5.3.11 Final System User Interface

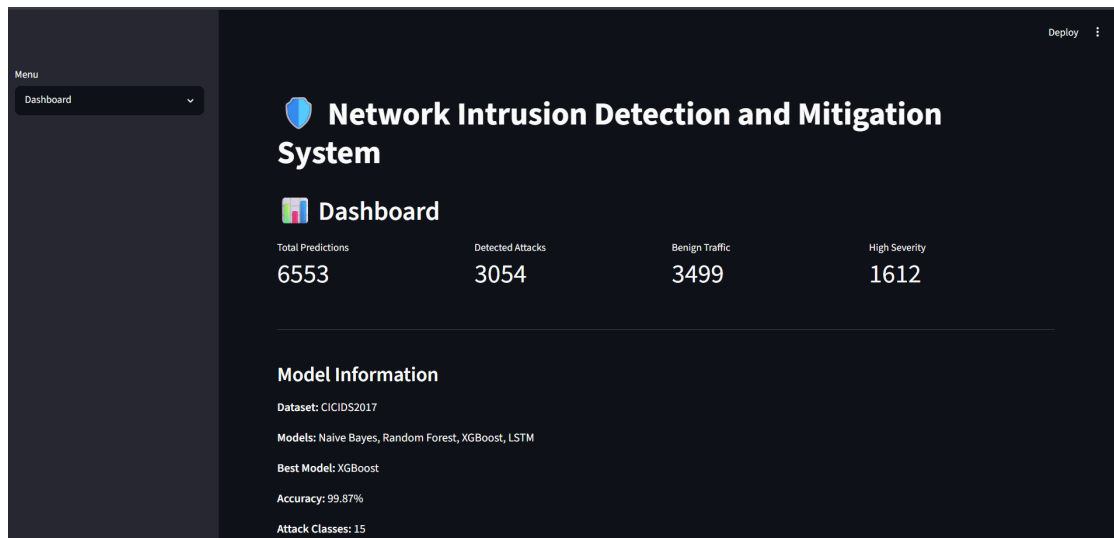


Figure 5.30: System User Interface

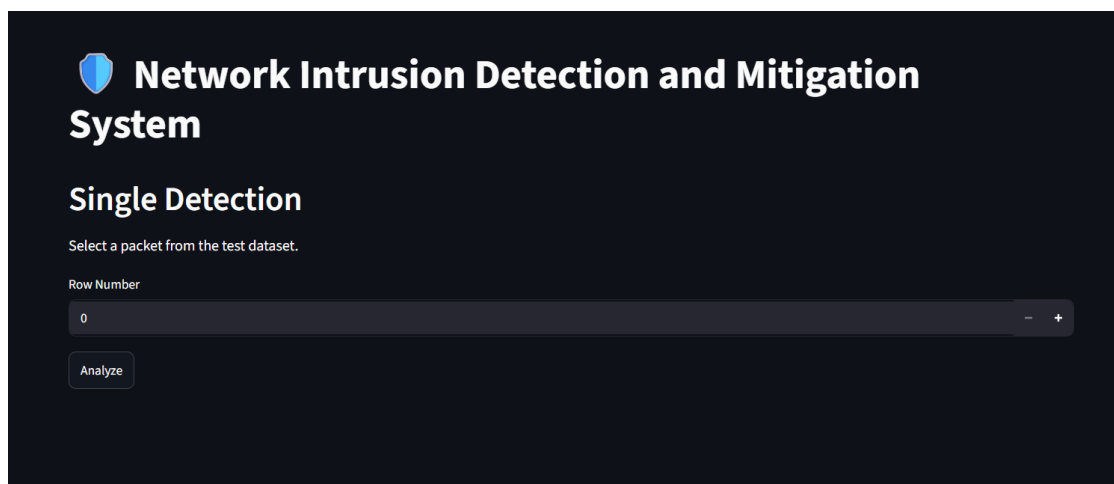


Figure 5.31: Single Attack Detection Interface

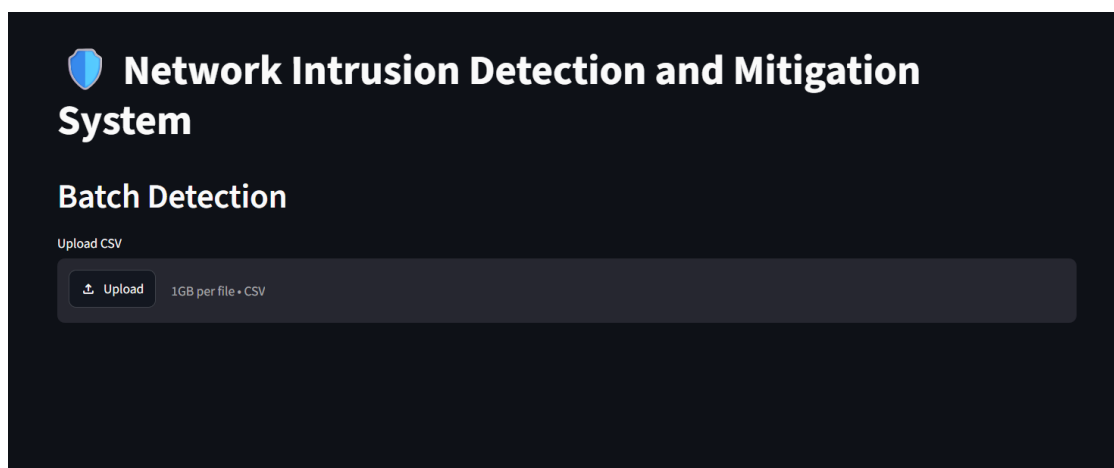


Figure 5.32: Batch Attack Detection Interface

6 Discussion and analysis

6.0.1 Discussion and Analysis

Model Accuracy and Detection Performance

The experimental results show that the ensemble-based models, Random Forest and XGBoost, significantly outperformed Naïve Bayes. Before applying SMOTETomek, XGBoost achieved an accuracy of 99.8687%, precision of 99.8648%, and F1-score of 99.8659%. After applying SMOTETomek, its performance improved slightly to 99.8792% accuracy, 99.8789% precision, and 99.8787% F1-score, making XGBoost the best-performing model.

Random Forest also showed a small improvement after SMOTETomek, with accuracy increasing from 99.8276% to 99.8552% and F1-score from 99.8194% to 99.8522%. In contrast, Naïve Bayes showed a slight decrease in performance, with accuracy dropping from 72.00% to 71.42%. Its lower performance can be attributed to its assumption that input features are independent, which is not well suited to the complex relationships present in network traffic.

The LSTM model achieved approximately 98.62% accuracy. SMOTETomek was not applied to the LSTM because it was evaluated using sequential network traffic data, where preserving the original temporal structure is important. Applying synthetic over-sampling directly could alter these sequential relationships. Therefore, LSTM was evaluated separately as a deep learning comparison rather than as part of the SMOTETomek-based experiments.

Effect of SMOTETomek

SMOTETomek produced different effects depending on the model. Both XGBoost and Random Forest showed improved accuracy and F1-score after preprocessing, while Naïve Bayes showed a small reduction. For XGBoost, accuracy increased from 99.8687% to 99.8792% and F1-score increased from 99.8659% to 99.8787%. This indicates that SMOTETomek provided a small improvement in the overall classification performance of the ensemble models.

Model Selection

Based on the comparative results, XGBoost with SMOTETomek was selected as the final model for the proposed NIDS. It achieved the highest overall performance, with 99.8792% accuracy, 99.8789% precision, and 99.8787% F1-score. The trained XGBoost model was subsequently integrated with the prediction and attack-mitigation components of the proposed system.

LSTM Training Performance

The LSTM model showed stable training and validation performance during its training process, achieving approximately 98.62% accuracy. The relatively close training and validation performance indicates that the model did not exhibit significant overfitting. However, its overall performance was lower than that of XGBoost and Random Forest. Since the LSTM was designed to process sequential traffic data, it was evaluated without SMOTETomek to preserve the temporal structure of the input data.

7 Conclusion and future enhancement

7.1 Conclusion

The proposed Network Intrusion Detection System (NIDS) demonstrated that machine learning can effectively identify malicious network traffic. Naïve Bayes, Random Forest, and XGBoost were evaluated before and after applying SMOTETomek, while LSTM was evaluated separately without SMOTETomek due to its sequential input requirements. Among the evaluated models, XGBoost with SMOTETomek achieved the best overall performance, with an accuracy of 99.8792%, precision of 99.8789%, and F1-score of 99.8787%. SMOTETomek slightly improved the performance of XGBoost and Random Forest, while Naïve Bayes showed comparatively lower performance. The LSTM model achieved approximately 98.62% accuracy, demonstrating the potential of deep learning for intrusion detection. Based on the comparative results, XGBoost with SMOTETomek was selected as the final model and integrated with the prediction and attack-mitigation components of the proposed NIDS. Overall, the results demonstrate that the proposed system provides a strong foundation for accurate network traffic classification and automated intrusion detection.

7.2 Limitations of the Project

The main limitations of the proposed system are:

- **Dataset Dependency:** Performance may vary on network environments different from CICIDS2017.
- **Limited Real-Time Testing:** The system has mainly been tested using pre-recorded network traffic.
- **Unknown Attacks:** The models may have limited ability to detect previously unseen attack patterns.
- **Class Imbalance:** Synthetic samples generated by SMOTETomek may not fully represent real network traffic.
- **Mitigation Limitation:** The system provides mitigation recommendations but does not automatically block attacks.

- **Computational Requirements:** XGBoost and LSTM may require higher computational resources for large datasets. ““

7.3 Future Enhancements

The proposed NIDS can be further improved in the following areas:

- **Dataset Evaluation:** Test the model on newer and unseen datasets to check its generalization.
- **Better Minority Detection:** Explore advanced techniques to improve detection of rare attack classes.
- **Real-Time Deployment:** Extend the system to detect attacks from live network traffic.
- **Online Learning:** Allow the model to learn from new attack patterns without full retraining.
- **Advanced Models:** Explore CNN-LSTM, GRU, and Transformer models for better detection.
- **Automated Mitigation:** Enable automatic blocking of malicious traffic and suspicious hosts.
- **Explainable AI:** Add explanations to help users understand the reasons behind predictions.
- **Performance Monitoring:** Continuously monitor model performance and re-train when needed.

7.4 Ethics of AI

AI tools were used during the development of this project as a supporting resource. They were mainly used to understand machine learning concepts, assist with code development and debugging, and improve the clarity and structure of the project documentation. AI-generated suggestions were reviewed, tested, and modified by the project

team before being incorporated into the system. The final implementation, model training, evaluation, and results were verified by the team to ensure their accuracy and reliability. AI was therefore used as an assistance tool rather than as a replacement for the team's own analysis and decision-making.

APPENDIX A

A.1 Report Schedule (Gantt Chart)

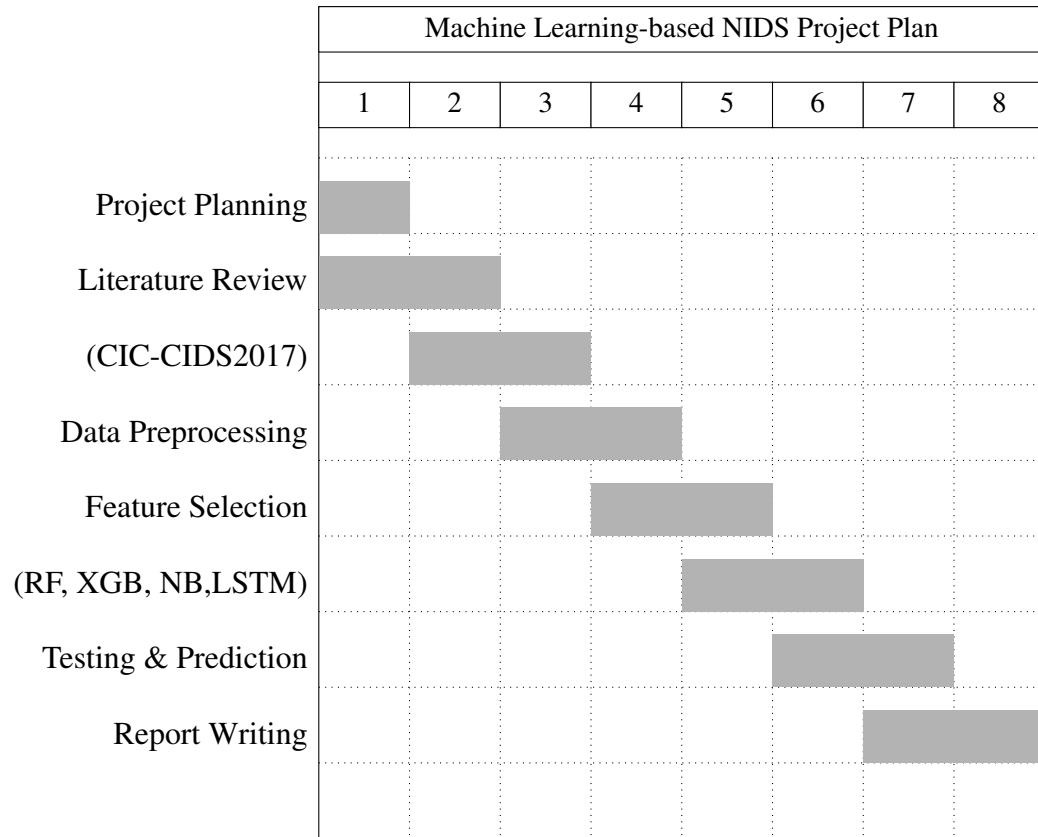


Figure A.1: 8-Week Gantt Chart for NIDS Project

REFERENCES

- [1] H. Hajjalian and C. Toma, “Network anomaly detection by means of machine learning: Random forest approach with apache spark.” <https://doi.org/10.12948/issn14531305/22.4.2018.08>, 2018.
- [2] E. Min, J. Long, Q. Liu, J. Cui, and W. Chen, “Tr-ids: Anomaly-based intrusion detection through text-convolutional neural network and random forest.” <https://doi.org/10.1155/2018/4943509>, 2018.
- [3] P. Maniriho and T. Ahmad, “Analyzing machine learning algorithms for ids.” <https://scholar.google.com/scholar?q=%22Analyzing+Machine+Learning+Algorithms+for+IDS%22>, 2018.
- [4] I. Obeidat, N. Hamadneh, M. Alkasassbeh, and M. Almseidin, “Intensive pre-processing of kdd cup 99 for network intrusion classification using machine learning techniques.” <https://arxiv.org/abs/1805.10458>, 2018.
- [5] W. S. Al-Yaseen *et al.*, “Machine learning for network intrusion detection—a comparative study.” <https://doi.org/10.3390/fi15070243>, 2023.
- [6] S. Sapre *et al.*, “Comparison of kddcup99 and nsl-kdd datasets.” <https://arxiv.org/abs/1912.13204>, 2019.
- [7] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks.” <https://doi.org/10.1109/ACCESS.2017.2762418>, 2017.
- [8] S. S. Tripathy and B. Behera, “Ml algorithms for ids performance evaluation.” <https://arxiv.org/abs/2310.00594>, 2023. arXiv:2310.00594.
- [9] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection.” <https://doi.org/10.1109/TETCI.2017.2772792>, 2018.
- [10] Çavuşoğlu, “A new hybrid approach for intrusion detection using machine learning methods.” <https://doi.org/10.1007/s10489-018-01408-x>, 2019.

- [11] C. I. for Cybersecurity, “Cicids2017 dataset.” <https://www.unb.ca/cic/datasets/ids-2017.html>, 2017.